

A comparison of different container sorting systems in modern rail-rail transshipment yards



Stefan Fedtke, Nils Boysen *

Friedrich-Schiller-Universität Jena, Lehrstuhl für Operations Management, Carl-Zeiß-Straße 3, 07743 Jena, Germany

ARTICLE INFO

Article history:

Received 13 January 2017

Received in revised form 7 June 2017

Accepted 15 June 2017

Available online 24 June 2017

Keywords:

Railway

Container logistics

Rail-rail transshipment yards

Sorter scheduling

ABSTRACT

Rail-rail transshipment yards act as central hub nodes within a railway network and enable a rapid consolidation of containers between different freight trains. To avoid an excessive movement of gantry cranes when transferring a container from one train to another, modern yards apply sorting systems where shuttle cars move containers horizontally along the spread of the yard. This paper compares four elementary sorting systems. Specifically, we compare rubber-tired and rail-mounted shuttles and differentiate whether a pure shuttle system or a lift & shuttle system is applied. In pure shuttle systems, a shuttle receives a container from a crane and transports it towards the destination crane, where it serves as a storage device until being unloaded. A lift & shuttle system applies shuttles with an integrated lifting platform, so that they are able to autonomously store and receive containers from a separate storage rack. Both alternatives exist in rubber-tired and rail-mounted versions. As the shuttles are the main driver of the investment costs, we compare the required fleet size for timely supplying given gantry crane schedules within all four systems. For this purpose, we derive suited scheduling procedures. This way, decision support for yard managers having to identify a suited sorting system and to layout a new terminal is provided.

© 2017 Elsevier Ltd. All rights reserved.

1. Introduction

Ever since 1955 when Delta Airlines pioneered the hub-and-spoke system by introducing its central Atlanta hub (Delta Air Lines, 2013), intermediate hub nodes have played a crucial role not only for the airline industry but also for alternative transportation devices. A hub-based consolidation of passengers and shipments in order to concentrate them along central transport relations is, for instance, also enabled by cross docks for a road based transportation (e.g., see Boysen and Fliedner, 2010). Another example are central ports where containers are exchanged between huge inter-continental container vessels and smaller feeder ships (e.g., see Zhen et al., 2011). For the railway network such a consolidation is equally essential, because otherwise only point-to-point transport between large urban areas that generate cargo volumes large enough to load full trains to capacity shows profitable (Trip and Bontekoning, 2002).

However, existing rail terminals are often ill-suited for a rapid transshipment of containers between freight trains. On the one hand, traditional shunting yards (e.g., see Boysen et al., 2012) require a time-consuming disassembly of inbound trains before they allow for a tedious new formation of trains by moving railcars over a shunting hill (Bontekoning and Priemus,

* Corresponding author.

E-mail addresses: stefan.fedtke@uni-jena.de (S. Fedtke), nils.boysen@uni-jena.de (N. Boysen).

URL: <http://www.om.uni-jena.de> (N. Boysen).

2004). Intermodal transshipment yards (e.g., see Boysen et al., 2013), on the other hand, are originally designed to exchange containers between freight trains and trucks by huge gantry cranes spanning rail tracks and truck lanes. If intermodal yards are beyond that also applied as hub nodes for transferring containers between different freight trains, then the resulting rail-rail container moves require long horizontal crane movements along the spread of the yard. This, in turn, leads to excessive crane interference on the cranes' shared track and, thus, prolonged train processing times.

To accelerate rail-rail container moves modern transshipment yards (also denoted as third-generation terminals, Boysen et al., 2013) apply an additional sorting system for moving containers horizontally along the spread of the yard. In a sorting system, a shuttle car moves to the horizontal position where a container is located on its incoming train, the container is loaded onto the shuttle by a gantry crane, and the shuttle moves towards the horizontal target position where the container is finally stacked on its dedicated position on an outbound train by another gantry crane. This way, cranes are unburdened from horizontal movements, so that obstructions among them are avoided and container processing is accelerated. This, in turn, makes the sorting system to the core element of the transshipment process and efficient shuttle operations to an important enabler of a rapid consolidation in rail-rail transshipment yards.

1.1. Sorting systems in modern rail-rail transshipment yards

The completion of such a modern rail-rail transshipment yard is currently planned for 2019 in Hannover-Lehrte (Germany). After a tedious planning phase over more than 16 years, which is documented by Aliche (2002) and Rotter (2004), the budget for this terminal has now been finally released and construction work has started. This terminal – also denoted as the *Megahub* – will consist of six rail tracks for processing freight trains by three parallel gantry cranes, which also span truck lanes (for parking and driving), an intermediate storage area for ground storage of containers, and a sorting system as is depicted in Fig. 1.

Preliminary simulation studies during the planning phase have identified a pure shuttle system consisting of 12 rail-mounted shuttle cars being a suited choice for the Megahub (Rotter, 2004). Currently rather a sorting system based on automated guide vehicles (AGVs) is preferred (Deutsche Bahn, 2016). In both systems, however, a shuttle is occupied by a container during the complete time span between the loading and unloading process of the cranes, so that a large number of shuttles is required. A separate container storage lane and shuttles with integrated lifting devices provided by a lift & shuttle system release shuttles from the storage function and therefore promise to get by with fewer shuttles. Another important distinction is whether rail-mounted shuttles or rubber-tired vehicles are applied. Among the rubber-tired vehicles there also exist pure shuttle systems, i.e., AGVs like they are commonly applied in large container ports (Steenken et al., 2004), and lift & shuttle systems (denoted as lift AGVs or ALVs, Vis and Harika, 2004). Fig. 2 schematically depicts the four resulting alternative sorter solutions. This paper aims at a general comparison of the four alternatives and their performance in rail-rail transshipment yards. All alternative systems are briefly characterized in the following (see also Kreutzberger and Konings, 2016):

- A1: In a *rail-mounted pure shuttle system*, a shuttle, i.e., some self-propelled railcar, is loaded with a container by a gantry crane (on a waiting bay next to the container's railcar on the inbound train), moves along a mono-directional, closed-loop rail track towards a waiting bay close to the container's destination, and waits there until another crane finally stacks the box onto its outbound train. In the initial plan for the Megahub, shuttles are designed to have rotatable wheels, which they turn by 90° when moving into a bay (Rotter, 2004).

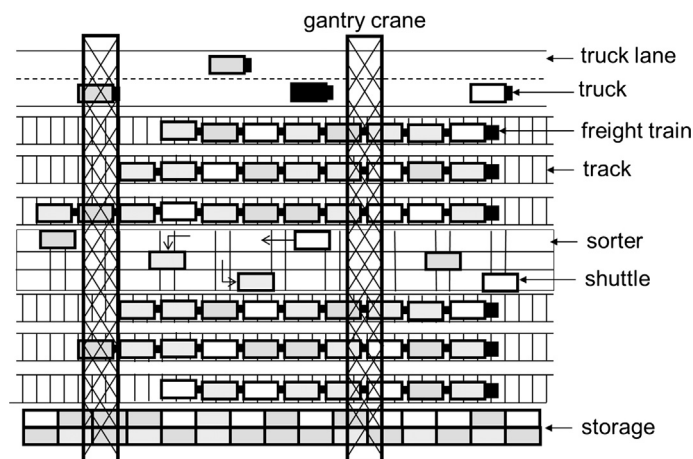


Fig. 1. Schematic layout of a modern rail-rail transshipment yard.

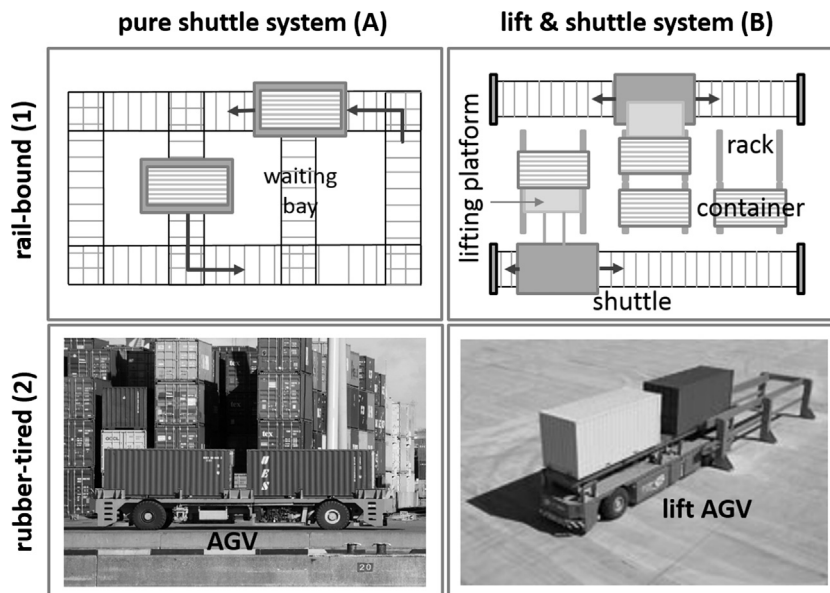


Fig. 2. Alternative sorting systems.

- B1: In a *rail-mounted lift & shuttle system*, the transport and storage functions are separated. Initially, a gantry crane sets down a container in a rack place of the sorter's storage lane. A rack place consists of simple steel base plates upholding a container from the ground, so that it becomes accessible from under the container by a movable lifting platform. This platform moves in some guide rails of the base plates and is part of a shuttle car. Thus, a shuttle car's integrated lifting platform allows them an autonomous storage and retrieval of containers. The shuttle inserts its lifting platform into a guide rail, retrieves a container from the storage lane, and moves along the sorter track towards the target railcar, where the container is stored into a free rack place of the storage lane. Finally, the container is retrieved from the storage lane by some gantry crane and loaded on an outbound train.
- A2: AGVs, which are often applied in container ports to move containers between quay cranes and container yards (Steenken et al., 2004), can also serve as *rubber-tired pure shuttle systems* in rail-rail transshipment yards. The only constructional prerequisite is a concrete area with guidepaths which can, for example, be wires in the ground or markings on the floor. More recent free-ranging technologies allow AGVs to operate without physical guidepaths. For a literature survey on AGVs see Vis (2006).
- B2: Lift AGVs enable a *rubber-tired lift & shuttle system*. They extend traditional AGVs by a mechanism to autonomously lift containers directly from and place them on a rack upholding a container from the ground (see Fig. 2). A more detailed description of these shuttles applied in container ports is provided by Vis and Harika (2004).

The main advantages of a rubber-tired AGV-based solution are the well-established technology (i.e., their introduction dates back to 1955, Vis, 2006) and the lower investment into fixed infrastructure. Rail tracks – especially if switches for turning the wheels in solution A1 are applied – have to be kept free of pollution, so that foliage and ice may require excessive cleaning effort especially during fall and winter. On the other hand, rail tracks reduce rolling resistance, so that energy consumption and noise exposure are reduced.

In a pure shuttle systems, a shuttle is occupied by a container move during the complete time span between the loading and unloading events of the two gantry cranes. Whenever these two events are not closely synchronized, excessive waiting time for shuttles threatens. Thus, shuttles also serve as storage-on-wheels, which increases the number of shuttles required. Lift & shuttle vehicles are relieved from the storage-on-wheels function and only occupied during container transport. This should allow a reduction of the shuttle fleet. On the other hand, the additional lift mechanism increases the investment costs per shuttle.

Note that especially the rail-mounted systems are merely representatives for their respective kind of sorting system and alternative layouts are possible. For instance, a pure rail-mounted shuttle system may also consist of only a single sorter track with evasion bays on the outside and a rail-mounted lift & shuttle system may also have only a single sorter track. However, the described layout for the former system is exactly the one initially planned for the Megahub and the latter layout equals an alternative sorter system evaluated during the planning phase of the same terminal project, so that two valid representatives for rail-mounted sorting should have been selected.

1.2. Decision problems and literature review

A recent survey paper summarizing all strategic and operational planning tasks arising in rail terminals is provided by Boysen et al. (2013). A review specifically addressing rail-rail terminals is authored by Kreutzberger and Konings (2016). For the modern rail-rail transshipment yards treated in this paper the following basic short-term decision problems (with basic papers treating the respective problem given in brackets) can be differentiated (see Boysen et al., 2013):

1. Schedule the service slots of trains by assigning them to bundles, which are successively processed (see Boysen et al., 2011; Boysen et al., 2012).
2. Assign each train a parking position on the rail tracks of the yard (see Aliche and Arnold, 1998; Kellner et al., 2012).
3. Determine the load plan of trains, i.e., the positions of containers on railcars (see Bostel and Dejax, 1998; Ambrosino and Siri, 2015).
4. Assign container moves to one of multiple gantry cranes (see Aliche, 2002; Boysen et al., 2010).
5. Determine the sequence of container moves per crane, where non-crossing constraints have to be considered (see Aliche, 2002; Martinez et al., 2004; Guo et al., 2013).
6. Schedule the shuttle cars in the sorter, such that the containers are timely supplied to gantry cranes (see Aliche, 2002).

This paper is concerned with the performance of four alternative sorting systems (A1, A2, B1, and B2 in Fig. 2), one of which has to be selected during the layout phase of a terminal. For evaluating the performance of a specific sorter, decision Problem 6 is to be solved. Shuttle scheduling, however, is closely related to the scheduling of the gantry cranes (Problems 4 and 5), because cranes and shuttles handover containers and, thus, need to be closely synchronized. However, the gantry cranes cause tremendous investment costs, which are much higher than those for the complete sorting system. Therefore, gantry cranes are typically the bottleneck resource in a terminal and a sorter should be equipped with enough shuttles, such that cranes never run idle. In our computational study we, thus, assume given crane schedules defining the start and end positions of containers along with arrival times and deadlines and we seek the minimum number of shuttles to find a feasible shuttle schedule not delaying the given gantry crane moves.

The scheduling of shuttles for a rail-mounted pure shuttle system A1 of the Megahub has already been integrated into the holistic planning approach of Aliche (2002). Here, a joint solution for Problems 4, 5 and 6 is sought, where, however, the sorter is not modeled in detail and no layout aspects are treated. The overall problem is modeled as a constraint satisfaction problem and different heuristic rules for fixing variables are compared. A further pre-study for the Megahub based on simulation, which exclusively integrates the rail-mounted pure shuttle system A1 and very basic decision rules, is provided by Meyer (1998).

The choice between rubber-tired pure shuttle and lift & shuttle systems equals the equipment choice in container ports for ground transport (see Carlo et al., 2014). Here, different studies present simulations studies and quantify the percentage of additional vehicles required in the AGV case compared to the lift AGV case to range between 38% (Vis and Harika, 2004) and 50% (Yang et al., 2004; Bae et al., 2011). However, the structure of container movements considerably varies between container ports and rail-rail transshipment yards. Therefore, these existing results are not directly transferable.

It can be concluded that a systematic comparison of alternative sorting systems in the rail terminal context based on sophisticated approaches for shuttle scheduling is yet missing.

1.3. Research question and paper structure

This paper aims to provide decision support for the layout phase of a terminal. During this phase, the right sorting system has to be identified and the selected system has to be dimensioned, i.e., an appropriately sized shuttle fleet has to be determined. To support this decision we generate representative scenarios consisting of given gantry crane schedules, which define where and when each container has to enter and leave the sorting system. Thus, we have arrival times and deadlines as well as start and target positions for containers and determine the minimum number of shuttles required within each sorting system for an accurately timed supply of containers. As the number of shuttle cars is the main driver for the investment costs of a sorting system, this way, a yard manager can weigh the structural pros of each alternative, e.g., reliability, against the investment cost. However, for appropriately dimensioning each system a detailed schedule for the shuttles is to be determined within each scenario. Therefore, we additionally provide suited scheduling policies for each kind of sorting system.

The remainder of the paper is structured as follows. To derive suited scheduling procedures, first, the pure shuttle systems are treated (Section 2). Both rubber-tired and rail-mounted pure shuttle systems can be represented by a unique model where only the parameters, e.g., for driving distances and velocities, have to be varied. Then, Sections 3 and 4 separately treat the rubber-tired and rail-mounted lift & shuttle system, respectively. After testing the computational performance of our scheduling approaches (Section 5), these scheduling policies are applied to a wide range of scenarios, so that the system performance of all four alternatives is derived (Section 6). This way, the shuttle fleets required for all sorting systems can be compared. Finally, Section 7 concludes the paper.

2. Pure shuttle systems

In spite of their very different outward appearance determining an appropriate fleet size for rail-mounted and rubber-tired pure shuttle systems (i.e., systems A1 and A2 in Fig. 2) can be modeled by the same scheduling approach. Before describing this approach we, first, characterize the container transfers within both systems in more detail.

The rail-mounted pure shuttle system (A1; see Fig. 3(a)) consists of a mono-directional system of rail tracks around the outline of the sorter for the shuttles to move on. This “outer loop” is rather a rectangle consisting of four straight rail tracks connected via switches. Whenever a shuttle reaches a switch at the corner of this rectangle, it has to rotate its wheels by 90° in order to cross over to the other track (Rotter, 2004). The loop is applied for the movement of shuttles whenever they transport a container from its given start to its target position or if they move empty towards the next job. Hereby, all shuttles travel counterclockwise on the loop in order to prevent interference with other vehicles. Container retrieval and delivery between a shuttle and the start and target gantry crane, respectively, take place in one of the waiting bays on the inside of the loop. For accessing a waiting bay, a shuttle also has to rotate its wheels and – once arrived – waits there for being either loaded or unloaded by a gantry crane.

The rubber-tired counterpart (A2) is depicted in Fig. 3(b). The concrete area is subdivided into three lanes: two mono-directional driving lanes in opposite directions on the outside and an inner parking lane. In the latter lane, AGVs receive containers from a crane and wait until being unloaded.

Both systems reduce interferences of shuttles to a minimum. A shuttle can only be blocked in the following two situations:

- A shuttle just having received a container in a waiting position wants to enter the loop or driving lane, but is blocked by another shuttle currently passing by. In this case, the blocked shuttle has to briefly wait for the passing before it can follow the other shuttle.
- A loaded shuttle wants to enter a waiting position, which is currently occupied by another vehicle. Instead of waiting for a release of the waiting position, so that other shuttle might get blocked, the shuttle should rather evade into a neighboring waiting position.

In both cases, the prolongation of container moves is rather short. If we, thus, completely neglect the occurrence of potential shuttle interference, we tend to underestimate the shuttle fleet. However, in light of the few sources for shuttle interference the loss in precision seems rather small. On the other hand, neglecting shuttle interference considerably facilitates to determine the required fleet of shuttles. Without interference determining the minimum number of shuttle cars equals a basic interval scheduling problem with sequence-dependent setup times (empty shuttle movement between two successive jobs) to minimize the number of machines (shuttles). An in-depth review on interval scheduling is provided by Kolen et al. (2007). Our specific interval scheduling problem for determining the shuttle fleet for the pure shuttle systems (rail-rubber-PSS) is defined as follows.

Let $J = \{1, \dots, n\}$ be the set of containers (jobs) to be processed by the sorting system. Each container move j is assigned an arrival time a_j and a deadline d_j defining the point in time a container is set down and picked up by the start and target crane, respectively. Furthermore, we have a sequence-dependent setup time $r_{jj'}$ specifying the uninterrupted travel time of a shuttle (on the shortest route) from the target waiting position of previous job j to the start position of successor job j' . Once, a job is assigned to a shuttle it is occupied during the complete time span between a_j and d_j . Recall that pure shuttle systems have no lifting ability and, therefore, have to wait for being processed by the cranes during the complete time span. Within rail-rubber-PSS, we seek a schedule Φ consisting of tuples $(i, j) \in \Phi$ defining processing of job j by shuttle i , such that:

- for each $j \in J$ there is exactly one (i, j) , that is each job is executed exactly once,

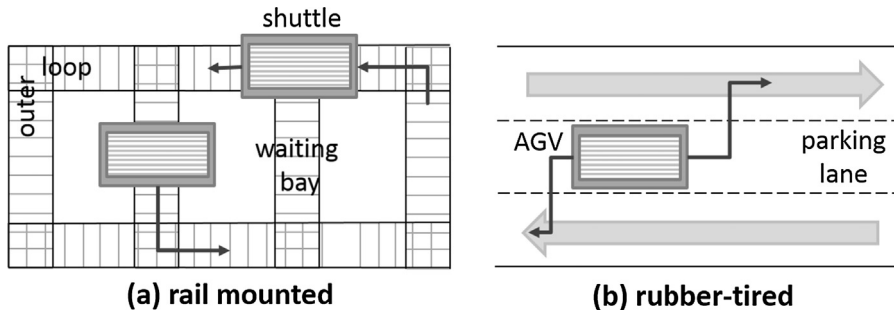


Fig. 3. Pure shuttle systems.

- for each pair of jobs $j, j' \in J, j \neq j'$ with $(i, j) \in \Phi$ and $(i', j') \in \Phi$ we have $i \neq i'$ or $d_j + r_{jj'} \leq a_{j'}$ or $d_{j'} + r_{j'j} \leq a_j$, that is jobs are executed by different shuttles or their processing intervals do not overlap, and
- $Z(S) = \max\{i | (i, j) \in \Phi\} \rightarrow \min$, that is the total number of shuttles $Z(\Phi)$ for processing all jobs without overlap is minimized.

The following theorem provides us with an efficient solution procedure for rail-rubber-PSS, so that we can quickly estimate an appropriate shuttle fleet for any given crane schedule.

Theorem 1. Problem rail-rubber-PSS can be solved in $\mathcal{O}(n^{2.38})$.

Rail-rubber-PSS is a basic optimization problem, which equals basic interval scheduling with sequence-dependent setup times (Kolen et al., 2007). The problem also resembles coloring/partitioning of interval graphs (e.g., Olariu, 1991), where, however, sequence-dependent setups are rarely considered. Our problem is well-known to be reducible to finding a maximum matching in a bipartite graph (Bertossi et al., 1987). A maximum matching in a bipartite graph can be found in $\mathcal{O}(n^{2.38})$ (Mucha and Sankowski, 2004). However, to keep the paper self-contained we summarize the reduction in Appendix A. Note that the same reduction to the maximum matching problem is applied in Boysen and Emde (2014) for minimizing the number of pickers replenishing the part demand of an assembly line.

With this efficient algorithm on hand the fleet sizes for given job sets can quickly be determined for rail-mounted and rubber-tired shuttle systems. The only difference arises when calculating sequence-dependent setup times $r_{jj'}$. When determining these parameters for a given set of crane moves the specific shuttle velocities and driving paths of both systems have to be considered, so that, nonetheless, differently sized shuttle fleets may be required.

3. Rubber-tired lift & shuttle systems

The setup of a rubber-tired lift & shuttle system applying lift AGVs is basically identical to that sorting system applying AGVs (see Fig. 4). On the outside of the concrete area are two driving lanes dedicated to opposite directions. The only difference is that lift AGVs can no longer receive and place containers anywhere along the parking lane, but are bound to metal racks upholding the boxes from the ground. The gantry cranes set down containers onto and pick them up from the metal racks, which are autonomously accessed by the lift AGVs. Therefore, lift AGVs do not have to wait for the cranes, but can deliver a box between the initial and target rack sometime during the given time interval.

Still, there are not many sources for shuttle interference, so that, again, we advocate to neglect interference. The loss in precision when estimating the required fleet size is outweighed by a much simpler optimization problem. Neglecting interference leads to a compact scheduling problem for determining the shuttle fleet for the rubber-tired lift & shuttle system (rubber-LSS), which can be defined as follows.

We have a set $J = \{1, \dots, n\}$ of jobs each representing a container transfer by shuttle between two gantry cranes. Each job has a given release time a_j , which is reached once the initial crane has set down the box in the respective rack position, and a deadline d_j . This deadline defines the latest time the box has to be delivered towards the dedicated target rack, so that the target crane can timely transfer the container onto the dedicated railcar. However, a job j does no longer occupy a lift AGV during the complete timespan from a_j to d_j , but the shuttle can autonomously pick up and deliver the container sometime during the given time window. Transferring a container from the start rack towards the target rack plus the respective pick-up and set-down operations take given processing times p_j . Furthermore, we have the sequence-dependent setup time r_{ij} defining the time span required for a lift AGV to move empty from the target rack of predecessor job i to the start rack of job j . Rubber-LSS aims at the minimum fleet size of lift AGVs, such that all jobs are executed within their given time windows.

Applying the notation summarized in Table 1 this scheduling task equals parallel machine scheduling with time windows where the number of machines is to be minimized. This problem can conveniently be formulated as a mixed-integer program. It consists of objective function (1) and constraints (2)–(7).

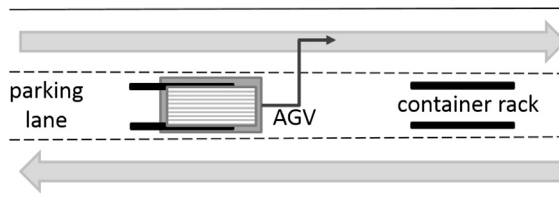


Fig. 4. Rubber-tired lift & shuttle system.

Table 1
Notation for rubber-LSS.

J	Set of jobs $J = \{1, 2, \dots, n\}$
p_j	Processing time of job j
a_j	Release date of job j
d_j	Deadline of job j
r_{ij}	Sequence-dependent setup time for moving from job i to j
M	Big integer, e.g., the end of the planning horizon
C_j	Continuous variable: completion time of job j
x_{ij}	Binary variable: 1, if job j is processed directly after job i by the same lift AGV; 0, otherwise
x_{0j}	Binary variable: 1, if job j is processed first by some lift AGV; 0, otherwise
$x_{i,n+1}$	Binary variable: 1, if job i is processed last by some lift AGV; 0, otherwise

$$(\text{Rubber-LSS}) \text{ Minimize } Z(C, X) = \sum_{j \in J} x_{0j} \quad (1)$$

$$\text{subject to } \sum_{i \in J \cup \{0\}} x_{ij} = 1 \quad \forall j \in J \quad (2)$$

$$\sum_{j \in J \cup \{n+1\}} x_{ij} = 1 \quad \forall i \in J \quad (3)$$

$$C_j \geq C_i + r_{ij} + p_j - M \cdot (1 - x_{ij}) \quad \forall i \in J \cup \{0\}, \quad j \in J \quad (4)$$

$$a_j + p_j \leq C_j \leq d_j \quad \forall j \in J \quad (5)$$

$$C_0 = 0 \quad (6)$$

$$x_{ij} \in \{0, 1\} \quad \forall i \in J \cup \{0\}, \quad j \in J \cup \{n+1\} \quad (7)$$

In our model, jobs assigned to lift AGVs are represented by job chains each started (ended) with virtual job 0 ($n+1$) and represented by binary variable x_{ij} . Thus, by minimizing the number of chains started objective function (1) minimizes the fleet size. Constraints (2) and (3) ensure that the chains are well defined, which means that each job has a predecessor and a successor. Properly spaced start times of jobs assigned the same lift AGV are ensured by (4) and constraints (5) enforce that all jobs are timely executed within their time intervals. Finally, completion time $C_0 = 0$ is ensured for the virtual start job and the domain of the binary variable is set by constraints (6) and (7), respectively. Note that some well-known valid inequalities, e.g., elaborated in [Unlu and Mason \(2010\)](#), could be added to this model. For reasons of clarity, however, we abstain from these extensions.

Unfortunately, machine scheduling with time windows is well-known to be strongly NP-complete ([Garey and Johnson, 1979](#)), so that rubber-LSS turns out being strongly NP-hard. Nevertheless, the model is quite handy and for our strategic research question computational times are a non-issue, so that in our computational study (Section 5) we try and solve our mixed-integer model with a standard solver.

4. The rail-mounted lift & shuttle system

4.1. Detailed description

Similar to its rubber-tired counterpart the rail-mounted lift & shuttle system separates transport and storage function. A container $j \in J$ is initially stored on an open rack place by the start gantry crane close to its position on an inbound train. The arrival time of container j in storage, i.e., the point in time it has just been set down by the start crane, is represented by a_j . Finally, we have the deadline d_j defining the point in time, at which job j has to be available at its predefined rack position, where it is picked up by the target gantry crane and transferred to the outbound train. Sometime in between a_j and d_j , a shuttle car has to transport the box, which means that the shuttle has to autonomously lift the container from its initial storage position, deliver it to the target position, and autonomously release the container into storage. The additional flexibility of separating transport and storage promises a smaller shuttle fleet compared to the pure shuttle system. However compared to its rubber-tired counterpart, the rail-mounted lift & shuttle system requires a much more complicated control of shuttle cars.

The rail-mounted lift & shuttle system (see [Fig. 5](#)) has two separate sorter tracks and each shuttle is fixedly assigned to either of them in order to move along the spread of the yard. As all shuttles of the same sorter track share their bi-directional pathway interferences have to be avoided and non-crossing constraints between shuttles have to be considered. Note that these shuttle interferences are the reason for having two sorter tracks, because unifying all shuttles on a single track would lead to even more obstruction. In general, collisions of shuttles in this system can be avoided in two different ways:

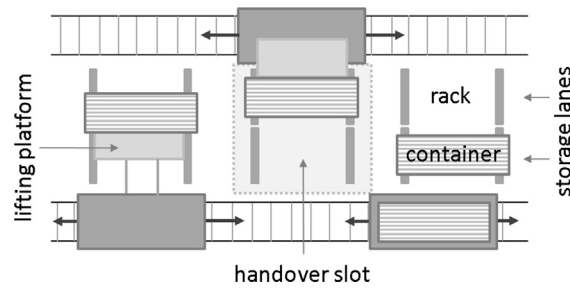


Fig. 5. Rail-mounted lift & shuttle system.

- If two shuttles meet and aim at a target position beyond each other, then one of them can evade and move back behind the target slot of the other shuttle. Once the prioritized shuttle finishes processing both can change direction and move back-to-back towards the target slot of the delayed shuttle.
- Especially longer container moves with far distant start and target slots require long evasion moves (potentially of even more than a single vehicle). Therefore, it might be advantageous to hand over containers between shuttles. This requires that one shuttle intermediately stores a box in an empty storage position and, afterwards, moves aside, so that another shuttle can retrieve the container later on and move it towards its target slot.

In this study, we only consider the latter type of avoiding interference. A major advantage for a container handover, which also caused the terminal managers to clearly prefer this solution, is that, this way, each shuttle can receive its unique area of operations, in which it can move independently. All container moves inside a shuttle's area are, then, directly processed by the assigned shuttle, whereas moves reaching over multiple areas are to be handed over at the area borders between neighboring shuttle cars. The complete storage area is then partitioned into separate shuttle areas, where two neighboring areas of the same sorter track do only overlap in a handover slot at their border. Handover slot are reserved for a container transfer among shuttles and, thus, not allowed to store containers for a shuttle-crane transfer. They can also be applied to transfer a container between both sides of the sorter, i.e., from one sorter track to the other. This fixed shuttle area solution has the major advantage that shuttle interference is reduced to a minimum and a complex online control system, which controls and optimizes the evasion moves of shuttles in realtime, becomes superfluous.

With the help of fixed shuttle areas the shuttle control is reduced to some very simple decision rules, which prioritize jobs simultaneously available. We will explain these rules in Section 4.4. However, on the superordinate decision level we still have to solve two planning problems:

- *Area sizing:* Partition the sorter into separate shuttle areas.
- *Container routing:* Determine the route between start and target position via handover slots for each container.

Both problems are challenging optimization problems by themselves and, moreover, heavily interdependent. Once shuttle areas are determined, the container routes via the handover slots determine which areas are passed and, thus, influence the workload of shuttles. If container routes are given, then varying the areas alters the shuttles' division of labor and, thus, their workload when transporting the boxes. Clearly, such a strong interdependence suggests to solve both problems jointly. A mixed-integer program defining this joint optimization task is given in Appendix B. However in the following two sections, we will, nonetheless, discuss how to solve both problems separately (once a solution for the other problem is given), so that they can iteratively be solved in an alternating manner. Our computational evaluation (see Section 5.2) comparing the holistic approach of Appendix B with our decomposition approach will show that the latter promises much better computational results.

This decomposition leaves us three more handy problems, but requires that on the superordinate decision level, i.e., when sizing the areas and determining containers routes, a surrogate objective is to be pursued, because the detailed routing of shuttle cars is determined not before the control level, where simple decision rules steer the movement of shuttle cars. As a surrogate objective, we aim to minimize the maximum workload of the loaded container moves among all shuttles. Whereas the delivery of containers from start to target rack is fixedly defined, the shuttles' empty movement between two jobs cannot be anticipated but depends on the detailed job sequence per shuttle and, thus, the control rules. Minimizing the maximum workload has the intention to level the workload among shuttles, so that each of them has an equal amount of buffer time for a timely supply of the gantry cranes. Testing whether our surrogate objective is indeed an appropriate aim will be part of our computational study in Section 5.

Algorithm 1. Decision framework for the rail-mounted lift & shuttle system

input: set of moves including arrival times, deadlines and processing times, setup times, set of handover slots

- 1 Set lower bound $LB := 2$ (one shuttle per track)
- 2 Set current fleet size $:= LB - 2$
- 3 **repeat**
- 4 Increase current fleet size by two and divide shuttles equally among tracks
- 5 // Solve area sizing and container routing in an alternating manner
- 6 Determine initial container routes by randomly selecting their handover slots
- 7 **repeat**
- 8 Solve the area sizing problem (see Section 4.2)
- 9 Solve the container routing problem (see Section 4.3)
- 10 **until** *maximum workload of shuttles has not changed during last iteration*
- 11 Determine the detailed shuttle movement by applying the control rules in a simulation run (see Section 4.4)
- 12 **until** *all containers are timely supplied*

return: current size of shuttle fleet

Given the three separate decision problems, we can pursue our actual aim – minimize the shuttle fleet while timely delivering all boxes – by the decision framework defined by Algorithm 1. At first, we set the shuttle fleet to a minimum of two shuttles, i.e., one per track. Subsequently, we check whether all containers can be timely supplied with the given fleet size. In order to do so, we iteratively optimize the shuttle areas and container routes in alternating manner for the current shuttle fleet. With each optimization, the maximum workload for shuttles decreases or remains constant. Once the maximum workload does not alter anymore, we hand fixed areas and container routes over to our simulation emulating shuttle control, which checks whether the available shuttle fleet can timely manage all given transport requests for the given routes and areas. If all containers are successfully executed, we found the smallest fleet size and abort our procedure. Otherwise, the fleet size is increased by an additional shuttle and another iteration is executed.

The following three sections treat each of the aforementioned three subproblems.

4.2. Area sizing

Consider a given set J of containers for which the routes through the sorter, i.e., the handover slots at which a box is transferred to the other sorter track, have already been defined. Without loss of generality we assume the sorter to go from left to right, so that storage positions $s = 1, \dots, S$ numbered according to their succession on the yard (also denoted as slots) can be applied to properly identify handover slots and storage positions of boxes. Furthermore, we have two storage tracks $t = 1, 2$ each servicing those rack places for container storage facing their track. With the help of slots and tracks the container routes, i.e., their initial and final storage positions and the handover slots they take, can precisely be defined. Each track t is assigned a given number δ^t of shuttles operating in separate areas each bordered by a handover slot. Handover slots $H \subset \{1, \dots, S\}$ reside at predefined positions and occupy m slots, which are without loss of generality assumed to be ordered according to their succession along the yard, so that $h_1 < h_2 < \dots < h_m$.

In this setting, we aim to minimize the maximum workload of loaded container moves among all shuttles. Recall that the shuttles' empty movement between jobs depends on their exact pathway in the sorter, which is not known when sizing the yard areas. It is easy to see, that once container routes are given, shuttles of separate tracks do not influence each other, so that we can solve the area sizing problem separately for both tracks $t = 1, 2$. For quantifying the resulting workload of a specific shuttle area we apply parameters $v_{h,h'}^t$ defining the workload of loaded moves for a shuttle serving a yard area between two handover slots located in slots h and h' on track t and $v_{1,h}^t$, resp. $v_{h,S}^t$, defining the workload between the first, resp. last, handover slot h and the beginning (slot 1), resp. end (slot S), of the yard. Note that these parameters are easily determined by adding up all moving times of loaded container moves going through this specific area, which are readily available due to the given container routes, plus the storage and retrieval times. Instead of a lengthy formal definition of these parameter calculations, we give a brief example:

Example. Consider the shuttle & storage system with $S = 9$ slots depicted in Fig. 6(a), where slots 3, 5, and 7 are handover slots. This sorter has to process the three depicted loaded moves $j = 1, 2, 3$ via the given routes. If any pick-up and set-down operation of a container is assumed to jointly last two time units and time is normalized such that the traversal of one slot requires one time unit, then we obtain the workloads stated in Fig. 6(b). The workload $v_{1,5}^1$ for an area ranging from slot 1 to the second handover slot on upper track $t = 1$ amounts to $v_{1,5}^1 = 10$, for instance. This includes the pick-up and set-down

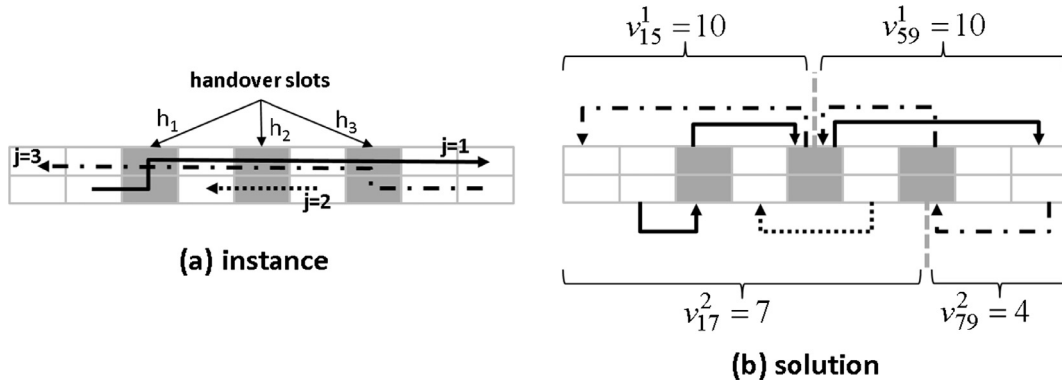


Fig. 6. Example for area sizing.

Table 2

Notation for the area sizing problem.

J	Set of container moves to be processed $J = \{1, 2, \dots, n\}$
S	Number of slots in the yard with $s = 1, \dots, S$
H	Set of handover slots with $H = \{h_1, h_2, \dots, h_m\} \subset \{1, \dots, S\}$, $h_1 < h_2 < \dots < h_m$ and $ H \geq \max\{\delta^1 - 1, \delta^2 - 1, 1\}$
t	Index of sorter track with $t = 1, 2$
δ^t	Number of shuttles, with $i = 1, \dots, \delta^t$, ordered according to their succession and assigned to sorter track t
$v_{h,h'}^t$	Workload (of loaded moves) for a shuttle serving the yard area between two handover slots located in slots h and h' on track t
$v_{1,h}^t$	Workload (of loaded moves) for a shuttle serving the yard area between the beginning of the yard and the first handover slot located in slot h on track t
$v_{h,S}^t$	Workload (of loaded moves) for a shuttle serving the yard area between the last handover slot located in slot h on track t and the end of the yard
x_i	Integer variable: position of the handover slot between the areas of shuttles i and $i + 1$

operations for containers $j = 1$ and $j = 3$ (4 time units in total) as well as the transport between the designated slots: Container $j = 1$ has to be moved 2 slots (2 time units) and container $j = 3$ has to be moved 4 slots (4 time units). Container $j = 2$ is not transported by this shuttle.

Applying the notation summarized in Table 2 our area sizing problem for the rail-mounted lift & shuttle system (rail-LSS-AS) for a track t can be defined as a mathematical program consisting of objective function (8) and constraints (9)–(12):

$$\text{(rail-LSS-AS) Minimize } Z^t(X) = \max_{i=1, \dots, \delta^t} \{v_{x_{i-1}, x_i}^t\} \quad (8)$$

$$\text{subject to } x_i > x_{i-1} \forall i = 1, \dots, \delta^t \quad (9)$$

$$x_0 = 1 \quad (10)$$

$$x_{\delta^t} = S \quad (11)$$

$$x_i \in H \quad \forall i = 1, \dots, \delta^t - 1 \quad (12)$$

By objective function (8) the maximum workload over all shuttles $i = 1, \dots, \delta^t$ assigned to sorter track t is minimized. The resulting workload for each shuttle i is determined by the workload v_{x_{i-1}, x_i}^t between its two handover slots at its left, i.e., x_{i-1} , and right, i.e., x_i , area border. Constraints (9) ensure that the shuttles are considered consecutively and enforce each shuttle

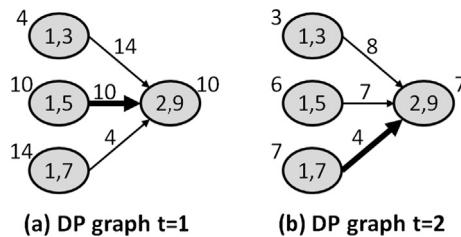


Fig. 7. DP graphs for our area sizing example.

to serve at least some part of the yard. Furthermore, it is ensured that the yard area of the first shuttle starts in the first slot (10) and last shuttle's area ends in the last slot (11). Finally, constraints (12) ensure that all areas are well defined and end in a handover slot.

The rail-LSS-AS for a track t can straightforwardly be solved in polynomial time by the following dynamic programming (DP) procedure. The DP is composed of δ^t stages where each stage represents a shuttle. Each stage contains states (i, h) defining that the right border of shuttle i 's area is handover slot h . The DP operates with a forward recursion, so that stage 1 (representing shuttle 1) has to be initialized by defining partial objective values $f_{1,h}$ for each possible right-hand border h :

$$f_{1,h}^t = v_{1,h}^t \quad \forall h = h_1, \dots, h_{|H|-\delta^t+2}. \quad (13)$$

Then, all remaining partial objective values can be calculated for each successive stage by a basic recursion as follows:

$$f_{i,h}^t = \min_{\substack{h_s \in H: \\ i-1 \leq s \wedge h_s < h}} \left\{ \max \left\{ f_{i-1,h_s}^t, v_{h_s,h}^t \right\} \right\} \quad \begin{array}{l} \forall i = 2, \dots, \delta^t - 1, \quad h = h_i, \dots, h_{i+m-\delta^t+1} \\ \text{and } i = \delta^t, \quad h = h_S. \end{array} \quad (14)$$

Solving rail-LSS-AS equals determining the objective value $f_{\delta^t,S}^t$ for the single state of final stage δ^t . Area borders for the optimal partition can be determined by a simple backward recursion along the states along the optimal path.

Example (cont.): Both resulting DP graphs for our example of Fig. 6(a) are depicted in Fig. 7. The solution resulting from the two bold faced optimal paths is the one depicted in Fig. 6(b) and the maximum workload over all shuttles of both tracks is 10.

There are at most m states in our DP within each of the δ^t stages and in the worst case all states of two successive stages are connected by a transition, so that the runtime complexity of our DP is in $\mathcal{O}(\delta^t \cdot m^2)$ and the problem is solvable in polynomial time.

4.3. Container routing

The container routing problem receives the shuttle areas of both tracks and the job set J of container moves as input data and aims to determine the pathway each box takes via the handover slots, such that the maximum workload of loaded moves is minimized among all shuttles. Each job has a given start and target position where it is set down and picked up by a gantry crane, respectively. As each rack place can only be serviced by a shuttle residing on the adjacent sorter track, we, thus, have fixedly defined which shuttle initiates and which finalizes each container move. In between these two steps each box can be handed over at varying handover slots between multiple shuttles, where each alternative defines one possible container route. Let R_j and $w_{i,t}^r$ define the set of alternative routes container j may take and the additional workload for the i th shuttle of track t once route r is selected, respectively. In Appendix C, we elaborate on how to determine (and restrict) the set R_j of potential optimal routes and, again, instead of a lengthy formal definition of how to determine workloads $w_{i,t}^r$ we give an example.

Example. Consider the sorter depicted in Fig. 8(a) with four shuttles, i.e., two per track, operating within their fixed areas (bordered by the dashed vertical lines). Again we assume that time is normalized such that the traversal of one slot takes one time unit and any pick-up and set-down operations of a container jointly take two time units. One potential route container $j = 1$ may take via the handover slots is given in Fig. 8(b). Initially, the first shuttle $i = 1$ of track $t = 2$, i.e., the bottom left one, moves the container by one slot between the start slot and the first handover slot, which takes $w_{1,2}^1 = 3$. Then the first shuttle $i = 1$ of track $t = 1$, i.e., the top left one, takes over and moves the box to its own area border ($w_{1,1}^1 = 4$), where the successive shuttle of the same track, i.e., the top right one, steps in and moves the box towards its final position ($w_{2,1}^1 = 6$). Analogously, the resulting shuttle workloads for all other routes can be determined.

Given all routes and their resulting workloads per area, by applying the notation summarized in Table 3 the container routing problem for the shuttle & storage system (rail-LSS-CR) can straightforwardly be formulated as a mathematical program consisting of objective function (15) and constraints (16) and (17):

$$(\text{rail-LSS-CR}) \text{ Minimize } Z(X) = \max_{t=1,2; i=1,\dots,\delta^t} \left\{ \sum_{j \in J} \sum_{r \in R_j} w_{i,t}^r \cdot x_{r,j} \right\} \quad (15)$$

$$\text{subject to } \sum_{r \in R_j} x_{r,j} = 1 \quad \forall j \in J \quad (16)$$

$$x_{r,j} \in \{0, 1\} \quad \forall j \in J, \quad r \in R_j \quad (17)$$

Objective function (15) minimizes the maximum (loaded) workload of all shuttles, where each shuttle car's workload is determined by adding up the workloads of all selected container routes falling into the respective area. Constraints (16) enforce that each container takes exactly one of its potential routes and (17) define the domain of the binary variables.

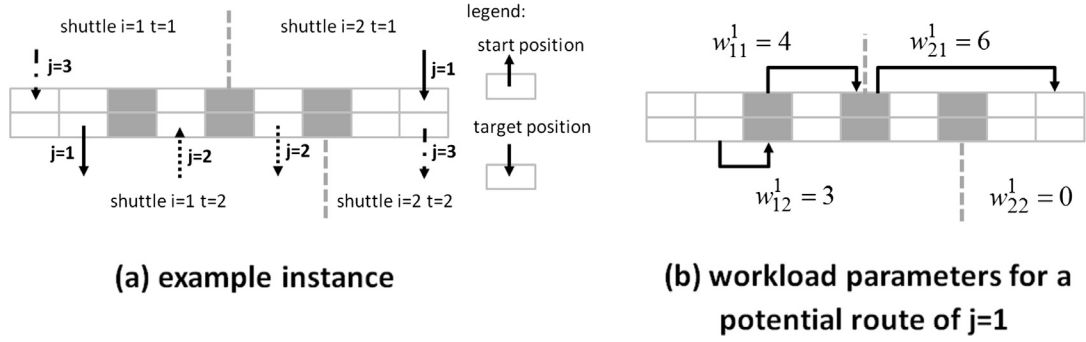


Fig. 8. Example for the container routing problem.

Table 3

Notation for the container routing problem.

J	Set of container moves to be processed $J = \{1, 2, \dots, n\}$
R_j	Set of possible container routes of container j
t	Index of sorter track with $t = 1, 2$
δ^t	Number of shuttles, with $i = 1, \dots, \delta^t$, ordered according to their succession and assigned to sorter track t
$w_{i,t}^r$	Workload (of loaded moves) caused by a container taking route r for the i th shuttle of track t
$x_{r,j}$	Binary variable: 1, if container j takes route r , 0 otherwise

Theorem 2. The rail-LSS-CR is NP-hard even if only a single shuttle serves each sorter track.

Proof. See Appendix D. $\square$

For solving our container routing problem we, now, introduce a DP scheme, which is extended by upper and lower bounds to a bounded DP in order to solve the problem to optimality. This scheme is, then, applied as the basis for a beam search heuristic.

The DP consists of $n + 1$ stages where each stage (plus virtual start stage zero) represents a job. Each stage contains states $(W_{11}, \dots, W_{\delta^1 1}; W_{12}, \dots, W_{\delta^2 2})$ defining the resulting workload for all shuttles per sorter track when processing the jobs up to the current stage. The (partial) objective value of each stage is calculated by

$$Z(W_{11}, \dots, W_{\delta^1 1}; W_{12}, \dots, W_{\delta^2 2}) = \max_{i=1, \dots, \delta^t; t=1, 2} \{W_{it}\}. \quad (18)$$

Starting from initial state $(0, \dots, 0; 0, \dots, 0)$ states are generated in a stage-wise manner by iterating through all $|R_j|$ potential successor states per predecessor state of previous stage $j - 1$. We, thus, simply evaluate each route R_j a job j may take and simply add the additional workload generated by this route r to the shuttles' previous workloads, so that a predecessor state

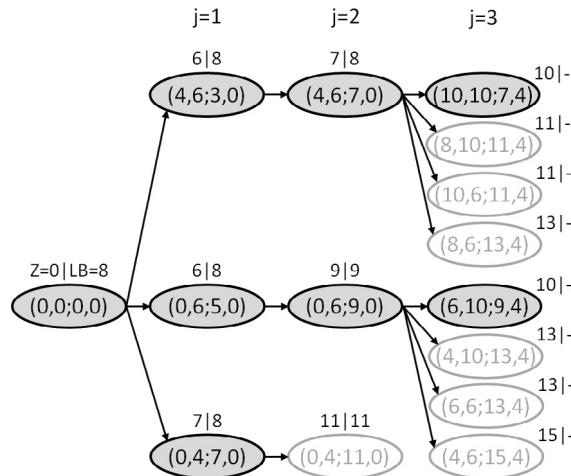


Fig. 9. DP graph for the container routing example.

$(W_{11}, \dots, W_{\delta^1 1}; W_{12}, \dots, W_{\delta^2 2})$ generates $(W_{11} + w_{11}^r, \dots, W_{\delta^1 1} + w_{\delta^1 1}^r; W_{12} + w_{12}^r, \dots, W_{\delta^2 2} + w_{\delta^2 2}^r)$. Clearly, an already existing state must not be regenerated, but only the transition has to be redirected to the existing state. Note that for identical states their partial objective value cannot vary, so that no update of this is required after redirecting a transition. This way, we generate the states of all stages, so that solving rail-LSS-CR equals determining one state of final stage n with minimal maximum workload. The routes each job should take can be determined by a simple backward recursion along the optimal path.

Within each stage we have at most $\bar{W}^\delta$ states, with $\bar{W}$ and δ being an upper bound on the maximum workload of a single shuttle and the total number of shuttles, i.e., $\delta = \delta^1 + \delta^2$, respectively. Even if all states of any two successive stages would be connected by a transition the total number of transitions is bounded by $n \cdot \bar{W}^{2\delta}$. Per transition an iteration through all shuttles is required, so that the total runtime of DP is in $\mathcal{O}(n\delta \cdot \bar{W}^\delta)$. Note that the runtime of DP is pseudopolynomial once the total number of shuttles is bounded by a given constant, so that rail-LSS-CR is proven to be ordinarily NP-hard for a constant shuttle fleet.

Clearly for instances of real-world size, the DP graph will grow to considerable size, so that we introduce two simple concepts to cut off parts of the graph without losing optimal solutions. First, the following straightforward dominance rule can be applied, so that dominated states need not be further developed and can be deleted for the DP graph.

Dominance rule: A state $(W_{11}, \dots, W_{\delta^1 1}; W_{12}, \dots, W_{\delta^2 2})$ dominates another state $(W'_{11}, \dots, W'_{\delta^1 1}; W'_{12}, \dots, W'_{\delta^2 2})$ of the same stage, if $W_{it} \leq W'_{it} \forall t = 1, 2, i = 1, \dots, \delta^t$.

Furthermore, we make DP to a bounded DP by introducing upper and lower bounds. Given these bounds all states which cannot improve an existing upper bound UB need not be further branched. An upper bound solution can be determined by the beam search approach described hereafter. Furthermore, a lower bound LB for the total path length is determined for any state. We elaborate on how this lower bound is derived in [Appendix E](#). Whenever this lower bound is equal or higher than upper bound UB , the respective state can be fathomed as it cannot improve the solution.

Example (cont.): The resulting bounded DP graph for our example of [Fig. 8\(a\)](#) and an upper bound solution of $UB = 10$ is depicted in [Fig. 9](#).

Even with these extensions, however, the number of states in the DP may of course very well still be exponentially large; finding an optimal solution may therefore take a long time for larger problems. In order to improve the running times, the DP can be turned into a beam search heuristic (BS, [Lowerre, 1976](#)). This requires defining a parameter ξ (the beam width) determining how many states per stage are further developed. To determine which states should be pruned in each stage of the algorithm, they are sorted in descending order according to their lower bound as calculated above and only the ξ best nodes are retained and further branched.

A first starting solution for the container routing problem can be obtained by assigning a simple route to each container. Hereby, *simple routes* do not include a transfer to another track, if the origin and the destination track of a container are the same and include exactly one transfer otherwise. The handover slot for a possible transfer is chosen randomly.

Given the solution procedures for the area sizing and the container routing problem both algorithms are executed in an alternating manner in order to minimize the maximum loaded workload among all shuttles. Fixed shuttle areas and container routes are then passed over to the real-time shuttle control, which is described in the following section.

4.4. Control (and simulation) of shuttle cars

Due to the fixed areas each shuttle is fixedly assigned to and the handover slots, which are applied to pass on containers at the area borders, the control rules required to avoid collisions of shuttles are very simple. It only has to be ensured that no collisions at handover slots occur and access to them is properly prioritized. These control rules are easy to be implemented in a real-world sorter, but also in a simulation study, which we apply to investigate whether a given shuttle fleet is able to timely process all containers for the given shuttle areas and container routes. The setup of this simulation (and the control of shuttles emulated by the simulation) is defined in the following by addressing three main aspects: the sorter layout, the container processing, and the movement of shuttles.

Sorter layout: The rail-mounted lift & shuttle system consists of two parallel storage lanes located in the middle of two rail tracks (see [Fig. 5](#)). As containers can only be placed on the given racks, the system can be partitioned horizontally into discrete slots. With the Megahub serving as a prototype for this study, we assume 100 container slots with a length of 7 m and a width of 3 m each, resulting in a total sorter length of 700 m. The given number of handover slots is distributed equispaced along the yard. Our study focuses sorter performance, so that we abstract from storage places being taken by containers stored there for a longer time span. For a matter of convenience, we assume that the storage area is empty and only consider containers received from the gantry cranes.

Container processing: All containers to be processed are assumed to have standard TEU size. An extension to different container sizes would be straightforward though. The simulation bases on discrete time units of 0.1 s. At any time, we check for gantry crane in- and outputs. If the current time equals the arrival time of a container, it is set into the designated starting slot. Once the deadline of an already executed sorter move is reached, the box is removed from the storage area. If containers have to switch tracks at a handover slot, we assume that the transfer is executed by the delivering shuttle. With regard to container storage we apply two limiting assumptions: First, we assume that multiple containers may be placed in the same

slot simultaneously. Furthermore, vehicles may pick any container from that slot disregarding their order. Even though these premises are certainly not suited for real-life application, they should be sufficient for estimating the shuttle fleet. Due to the following reasons these premises are assumed to not overly falsify our results: If a storage position is blocked, a shuttle would just unload its container into a neighboring slot and not wait for the container to be removed. Also, the considered system is supposed to have several handover slots next to each other. In this case, each “handover section” can process several containers simultaneously.

Vehicle movement: At the start of our simulation, each vehicle is positioned empty in the middle of its designated area. Once a container is set down into a shuttle’s area (either by crane or another shuttle at a handover slot), it is added to the shuttle’s task list. If the task list of a vehicle is empty, it either waits in the current position or evades one slot to the side, if it blocks a handover slot on the borders of its area. This way, shuttle interference at handover slots is reduced beforehand. Nevertheless, it may occur that a handover slot is blocked by a vehicle. In this case, other shuttles have to wait, until the slot is cleared. If two shuttles simultaneously aim to enter a handover slot the shuttle carrying the more urgent container is preferred. If a shuttle’s task list contains multiple jobs, the next container to be processed is chosen with respect to the earliest deadline.

We execute this simulation until all container moves are processed and delivered towards their dedicated cranes.

5. Setup of computational study and preliminary tests

Before we can apply our different scheduling approaches to compare the resulting fleet sizes for all alternative sorting systems, we elaborate on the setup of our computational study. First, we address the generation of test instances in Section 5.1. Furthermore, we have to show that our solution procedures derived for the rail-mounted lift & shuttle system are in-deed well suited. To do so, we benchmark our decomposition procedure with a holistic approach solving the area sizing and container routing problem simultaneously in Section 5.2.

5.1. Instance generation

To perform our computational experiments we have to generate test instances. Hereby, a key task is to derive instances usable for each sorting system.

Table 4 lists the parameters used for instance generation. Given these values, we can construct a data instance, which basically consists of a set of container moves (jobs) with given storage positions and time windows. Recall that the rail-mounted lift & shuttle system includes two storage lanes and possible lane transfers have to be considered whereas the other systems only include one storage/loading lane. Therefore, the probability for container transfer α only matters in system B1. We generate each job $j = 1, 2, \dots, |J|$ randomly by determining the following two components:

Storage positions for the container origin $(lane_j^o, slot_j^o)$ and destination $(lane_j^d, slot_j^d)$ have to be determined, each including a lane and a service slot. Note that storage lanes can be discarded if dealing with systems A1, A2 and B2. First, we draw four numbers RND_1, RND_2, RND_3 and RND_4 from the interval $[0, 1)$ randomly. The tracks are now determined by

$$lane_i^o = \begin{cases} 1 & , \text{if } RND_1 < 0.5 \\ 2 & , \text{otherwise} \end{cases}, \quad lane_j^d = \begin{cases} 3 - lane_j^o & , \text{if } RND_2 < \alpha \\ lane_j^o & , \text{otherwise} \end{cases}, \quad (19)$$

which considers the probability for a container transfer to the other side of the sorter’s storage area α . In order to generate service slots, we first draw the job’s length l_j from the given interval L randomly and then determine the slots according to l_j :

$$\begin{aligned} slot_j^1 &= Rd(RND_3 \cdot (99 - l_j)) + 1 \\ slot_j^2 &= slot_j^1 + l_j \end{aligned}, \quad (slot_j^o, slot_j^d) = \begin{cases} (slot_j^1, slot_j^2) & , \text{if } RND_4 < 0.5 \\ (slot_j^2, slot_j^1) & , \text{otherwise} \end{cases}, \quad (20)$$

where $Rd(x)$ defines the next integer value to x .

Time windows: To compare the sorting systems, we need to ensure, that each system can process a job within its given time window. Therefore, the length of job j ’s time window has to be greater than the processing times of job j within each system.

Table 4
Parameters for instance generation.

Symbol	Description
$ J $	Number of jobs (containers)
L	Interval for the length of job (in slots)
α	Probability for container transfer
β	Length of time windows
T	Time span

Table 5

Moving and un-/loading times of AGVs.

Symbol	System	Description	Values (s)
tS^{A1}	Pure-rail	AGV time to move one slot	2.3
tV^{A1}	Pure-rail	AGV time for vertical movement	15.0
tS^{A2}	Pure-rubber	AGV time to move one slot	1.2
tV^{A2}	Pure-rubber	AGV time for vertical movement	1.0
tS^{B1}	Lift-rail	Shuttle time to move one slot	1.6
tL^{B1}	Lift-rail	Shuttle time to un-/load a container	35.0
tS^{B2}	Lift-rubber	Lift AGV time to move one slot	1.2
tV^{B2}	Lift-rubber	Lift AGV time for vertical movement	1.0
tL^{B2}	Lift-rubber	Lift AGV time to un-/load a container	22.0

Given the sorting systems' specifications in Table 5, which are realistic parameter values either specified by parties involved in the Megahub project or from a producer of AGV systems (Gottwald, 2016), we first determine the minimal processing times to execute job j in each system

$$p_j^{A1} = l_j \cdot tS^{A1} + 2 \cdot tV^{A1}, \quad (21)$$

$$p_j^{A2} = l_j \cdot tS^{A2} + 2 \cdot tV^{A2}, \quad (22)$$

$$p_j^{B1} = l_j \cdot tS^{B1} + 2 \cdot tL^{B1} + 2 \cdot tL^{B1} \cdot |\text{lane}_i^o - \text{lane}_i^d|, \quad (23)$$

$$p_j^{B2} = l_j \cdot tS^{B2} + 2 \cdot tV^{B2} + tL^{B2}. \quad (24)$$

These times include the moving time of the vehicles (A1, A2, B1, B2), the time for vertical movement from parking lane/waiting bay to the driving lane (A1, A2, B2), the loading and unloading time (B1, B2) as well as the (potential) additional un-/loading time needed to transfer a container between lanes (B1). Hereby, we assume, that all shuttles of the same system move with equal speed and neglect acceleration. Note that, analogously, the sequence-dependent setup times r_{ij} , i.e., the empty movement of a vehicle between jobs i and j , can be calculated. The minimal length of the job j 's time window equals the maximum of the processing times in all four systems: $p_j^{\min} = \max \{p_j^{A1}, p_j^{A2}, p_j^{B1}, p_j^{B2}\}$. The length of this time window can be extended to add possible flexibility by multiplying the processing time with factor $\beta \geq 1$. Finally, the time window, including arrival time a_j and deadline d_j can be drawn randomly, given a random number RND_5 from $[0, 1)$ and the length of the observed time span T

$$a_j = Rd(RND_5 \cdot (T - \beta \cdot p_j^{\min})), \quad d_j = a_j + \beta \cdot p_j^{\min}. \quad (25)$$

We repeat this process for each job $j = 1, 2, \dots, |J|$ and obtain an instance feasible for each sorting system. For all data instance we fix the relevant time span T to 1200 s = 20 min. All procedures were implemented in Visual Basic and tests were performed on a 64-bit system on a Intel(R) Core(TM)2 Quad CPU with 2.83 GHz and 8 GB memory. As a standard solver we apply Gurobi optimizer (version 6.5).

5.2. Computational performance of the decomposition approach for the rail-mounted lift & shuttle system

For determining the shuttle fleet within the rail-mounted lift & shuttle system we introduced a decomposition approach (dubbed DEC) in Section 4. This section is dedicated to testing the computational performance of DEC. DEC solves the sizing of the areas served by dedicated shuttle cars (rail-LSS-AS) and container routing via the handover slots (rail-LSS-CR) in an alternating manner. We compare the performance of DEC with the holistic approach, which solves both problems simultaneously. The resulting mixed-integer program for the holistic approach is defined in Appendix B. This mixed-integer program is solved with standard solver Gurobi and benchmarked with DEC.

Table 6

Parameters for instance generation.

Symbol	Description	Values	
		Small	Large
$ J $	Number of container moves	5, 6, ..., 15	20, 30, 40, 50
L	Interval for the length of moves	[10; 90]	
α	Probability for container transfer	0.5	
β	Length of time windows	1, 2, 5	
$ H $	Number of handover slots	3, 6, 9	
δ^t	Number of shuttles on track t	1, 2, 3, 4, 5	

Table 7

Computational performance for small instances.

J	Gap [%]				Opt [%]				cpu time [s]				
	RND	DEC 100	DEC 250	DEC 500	RND	DEC 100	DEC 250	DEC 500	RND	DEC 100	DEC 250	DEC 500	GRB
5	76.40	−2.91	−2.78	−2.84	3.61	75.90	76.51	75.90	<0.01	0.15	0.41	0.64	84.56
6	81.25	−4.71	−4.89	−5.21	0.62	73.29	75.16	81.37	<0.01	0.26	0.54	0.95	96.19
7	75.47	−5.40	−5.77	−5.96	0.64	76.43	73.89	76.43	<0.01	0.31	0.62	1.56	104.95
8	84.82	−5.03	−4.76	−4.98	0.66	75.66	71.05	71.71	<0.01	0.29	1.09	1.90	112.05
9	83.38	−4.67	−5.11	−5.63	2.24	73.88	76.87	77.61	<0.01	0.50	1.82	4.11	128.38
10	82.97	−4.19	−4.02	−4.44	0.74	77.04	73.33	77.04	<0.01	0.81	2.57	9.58	132.45
11	82.27	−4.34	−4.90	−5.33	1.54	76.15	76.92	77.69	<0.01	0.80	3.52	6.90	134.05
12	77.13	−4.45	−4.85	−5.19	0.00	70.50	74.82	75.54	<0.01	0.86	3.18	8.96	130.69
13	81.60	−4.92	−5.51	−5.59	0.00	74.42	77.52	78.29	<0.01	0.66	2.97	6.80	139.70
14	77.81	−5.58	−6.89	−6.63	0.00	79.13	80.87	79.13	<0.01	1.24	4.55	10.56	148.59
15	79.05	−5.35	−6.07	−6.57	0.82	75.41	77.87	78.69	<0.01	1.21	4.92	14.07	142.79
Avg	80.20	−4.69	−5.05	−5.30	1.04	75.19	75.71	77.14	<0.01	0.64	2.38	6.00	123.11

Within our experiments we distinguish two cases: small instances easier to solve by an off-the-shelf solver and larger instances representing real-world problems. Table 6 specifies the parameters utilized for data generation.

For each combination of parameters $|J|, L, \alpha$ and β instance generation (see Section 5.1) is repeated 5 times leading to a total of 165 small and 60 large instances. Furthermore, we build several settings for the sorting system by applying different numbers of handover slots $|H|$ and shuttles on each rail track δ^t . As handover slots are mandatory in between two different shuttle areas on the same track, we only consider settings with $|H| \geq \max\{\delta^1 - 1, \delta^2 - 1, 1\}$. Consequently, 14 alternative sorter settings are considered for each instance.

At first, we investigate the performance of DEC for the small instances. Our beam search procedure applied for solving the container routing (see Section 4.3) is executed with different beam widths $\xi \in \{100, 250, 500\}$. Standard solver Gurobi (GRB) is executed with a time limit of 300 s (900 s) for small (large) instances within each scenario. As a further benchmark we add the results of random solutions (RND) where areas and container routes are randomly determined. Table 7 shows the relative gap of the objective value between GRB and the other procedures (in %) and the cpu runtimes (in seconds) for each solution approach. It also lists the percentage of optimal solutions found by the heuristic procedures among those scenarios Gurobi was able to determine the optimal solution.

Within a given time limit of 300 s Gurobi is not able to find optimal solutions in each case. Among all 2310 scenarios only 1540 are solved to optimality by the standard solver. Especially instances with a larger number of moves or handover slots seem to trouble Gurobi. The solutions determined by DEC, on the other hand, lead to better objective values on average, which is shown by the negative gaps in columns 3–5. Moreover, DEC is significantly faster than Gurobi. However, DEC is not able to always find better solutions than Gurobi. Among the 1540 scenarios solved to optimality by Gurobi, DEC found optimal solutions in between 75.19 and 77.14% cases depending on the beam width. Especially some smaller instances are solved to optimality by Gurobi, but not by DEC due to the decomposition of the problem. Still, the optimality gap is not large. When facing larger instances, Gurobi's solutions are worse than those of DEC. In total, DEC outperforms Gurobi by means of quality and runtime. Drawing solutions randomly provides a very fast alternative. Run times are barely measurable, i.e., below 0.01 s on average, but the optimality gaps are considerable.

These trends continue for the larger instances (see Table 8). Within the given time frame of 900 s, Gurobi barely found optimal solutions. It performs much better than the random approach (average gap of 80.19 %), but is clearly outperformed by decomposition approach DEC both with regard to solution quality (i.e., the average gap at a beam with of $\xi = 500$ amounts to 6.3%) and solution time (i.e., even with the largest beam width of $\xi = 500$ DEC is about 14 times faster).

When applying the decomposition approach, the area sizing problem and the container routing problem are iteratively solved, until the maximum workload does not improve anymore. With reference to the repetitions in this loop (lines 6–9 in Algorithm 1), it can be stated that the final solution is determined within very few iterations. For small and large instances, the number of repetitions varies from 1 to 5 with an average of 2 repetitions for the area sizing and 1 repetition for the container routing problem.

Table 8

Computational performance for large instances.

J	Gap [%]				cpu time [s]				
	RND	DEC 100	DEC 250	DEC 500	RND	DEC 100	DEC 250	DEC 500	GRB
20	87.89	−1.00	−2.23	−2.98	<0.01	1.55	6.02	17.86	447.94
30	82.10	−4.01	−5.14	−6.01	<0.01	2.77	12.94	39.75	510.48
40	77.63	−5.24	−6.34	−6.87	<0.01	3.86	15.81	41.42	553.92
50	73.10	−7.57	−8.35	−9.37	<0.01	4.79	15.84	54.59	616.43
Avg	80.19	−4.45	−5.51	−6.30	<0.01	3.24	12.65	38.39	532.09

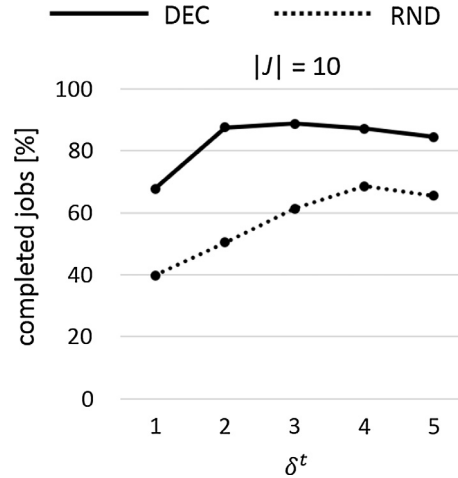


Fig. 10. Fraction of completed jobs for the rail-mounted lift & shuttle system.

All in all, DEC provides an adequate approach to solve the combined area sizing and container routing problem (rail-LSS) even for larger instances. Note that all further experiments, thus, apply DEC with a beam width of $\xi = 250$, which showed as a good compromise between runtime and solution quality.

Up to now, we have only shown that our decomposition procedure outperforms the holistic approach, which tackles area sizing and container routing simultaneously. However, both approaches follow a surrogate objective, i.e., they minimize the maximum loaded workload among all shuttles. To ensure that this surrogate objective indeed contributes to more efficient sortation processes, we generate a further set of instances with $|J| \in \{10, 20, \dots, 50\}$ and all other parameters as listed in Table 6. For each combination, we generate 100 instances and determine two solutions for each of them, a random one and another one determined with decomposition approach DEC. Subsequently, both solutions are simulated. This way, the exact number of completed jobs timely supplied within our given planning horizon of 20 min can be computed. The results are averaged over the 100 repetitions and analyzed for each procedure. The results of this test for $|J| = 10$ jobs are depicted in Fig. 10. In this figure the percentage of completed jobs is related to the number of shuttles δ_t per sorter track of the rail-mounted lift & shuttle system. Note that a dependent t -test for paired samples showed that the results of DEC are significantly better than random solutions at a significance level of 0.01.

The results show that our optimization approach (and the surrogate objective it follows) contributes to more efficient sortation processes. The fraction of completed jobs is considerably higher than that achieved by the random approach. Very similar graphs occur for other job numbers as well. Therefore, optimizing the shuttle areas and container routes is an important lever to improve the sortation efficiency of a rail-mounted lift & shuttle system. Similar results occur when applying other measurements in order to evaluate the performance of the yard. We executed the same experiment for two other objectives, namely the ‘completion time’ of all container jobs and the average ‘tardiness’ for container pick-ups by the gantry crane (both in seconds). For each measure, we achieved significant improvements when applying our decomposition approach instead of random solutions. Whereas the ratio of completed jobs increased by 26.03% (absolute value) on average, as stated in Fig. 10 for $|J| = 10$ jobs, the completion time decreased by 28.20% (relative value) and the tardiness decreased by 64.65% (relative value) on average. For the sake of simplicity, we confine ourselves to the ratio of completed jobs as a measure for all further experiments.

Fig. 10 reveals another interesting property of the rail-mounted lift & shuttle system: Increasing the number of shuttles does not necessarily improve the sortation performance. Most jobs are completed with $\delta_t = 3$ shuttles on each of both sorter tracks. If more shuttles are added, this means that the dedicated areas of each shuttle become smaller, so that more handovers are required when moving containers through multiple areas. Each additional handover slows down the container transfer, so that more boxes arrive delayed at their target positions. Therefore, sortation performance of the rail-mounted lift & shuttle system deteriorates if too many shuttles are added to the system.

This property, however, reveals that our comparison of the alternative sorting systems with the four introduced scheduling approaches for determining the respective fleet sizes is unfair for the rail-mounted lift & shuttle system. The rail-mounted lift & shuttle system (B1) is the only one where shuttle interference is avoided with certainty. By introducing dedicated shuttle areas connected via handover slots and the control rules detailed in Section 4.4 no shuttle interference occurs. The price for this is that the number of shuttles that can be (reasonably) added to the system is restricted. For all three other sorting systems we presuppose by assumption that no interference occurs. This (theoretically) allows us to add a very large shuttle fleet to the system, so that any workload of jobs can (theoretically) be handled just by adding additional shuttles to the fleet. In the real world, however, the probability of shuttle interference increases the more shuttles are added to a sorter. Recall that ruling out shuttle interference leads to much simpler optimization approaches. However, to balance the struc-

Table 9

Parameters for instance generation.

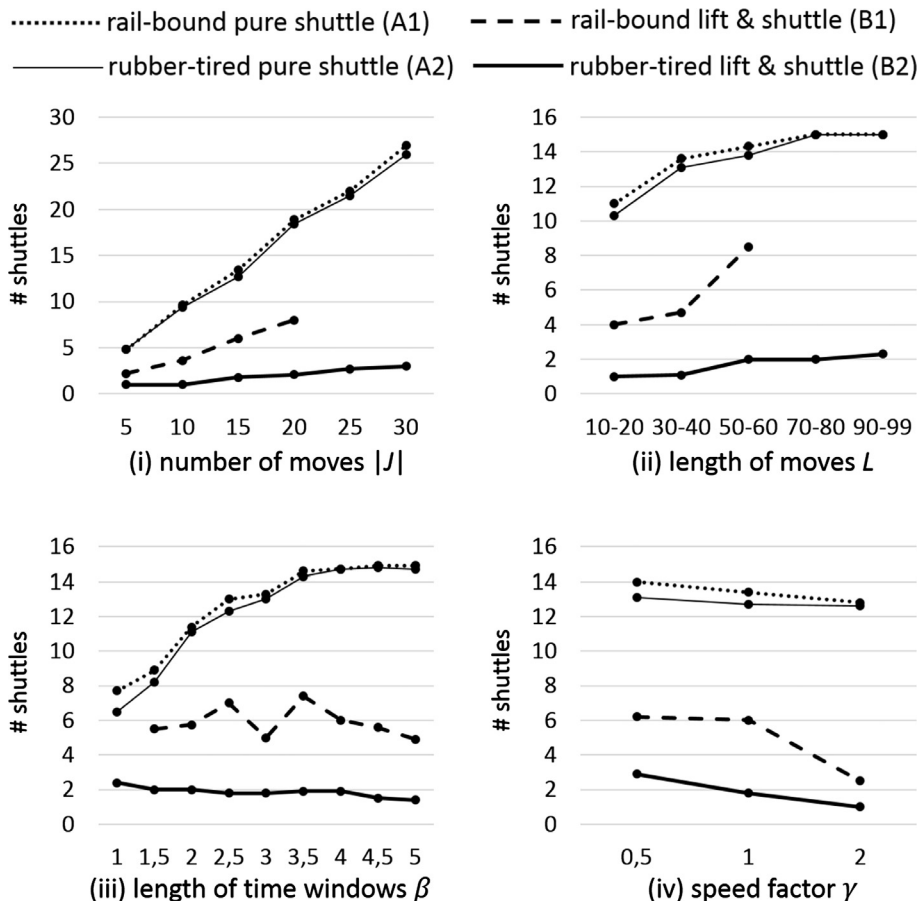
Symbol	Description	Values	Default value
$ J $	Number of container moves	5, 10, 15, 20, 25, 30	15
L	Interval for the length of moves	[10; 20], [30; 40], ..., [90; 100]	[10; 90]
α	Probability for container transfer	0.0, 0.1, 0.2, ..., 1.0	0.5
β	Length of time windows	1.0, 1.5, ..., 5.0	3.0
γ	Speed factor	0.5, 1.0, 2.0	1.0

tural disadvantage of the rail-mounted lift & shuttle system (B1) we say that this system has timely supplied a given workload of jobs, if 90% of all jobs arrive on time. Small delays of 10% of the boxes are, thus, allowed. The other three systems have to deliver all job punctually. Note that this value has been agreed among a handful of experts engaged with the Megahub project as a fair and realistic compensation. In the following section we detail our comparison of the four alternative sorting systems.

6. Comparison of alternative sorting systems

This section is dedicated to our basic research question: the comparison of the four alternative sorting systems. Specifically, we quantify the sizes of the shuttle fleets required by each system to timely supply containers for different workloads. Recall that the shuttles are the main cost component of a sorting system, so that this way practitioners receive some decision support when having to design a specific sorting system.

For our experiments, we apply our instance generator described in Section 5.1 and hand over the parameter values listed in Table 9. Hereby, we confine the number of container moves to a range of 5–30 moves. These values are derived from the goal of the Megahub operators of achieving a sortation performance of 18 moves within 20 min. As a new parameter we introduce γ , which varies the processing speed for each system. A value of $\gamma = 2$ means, that the speed of the system doubles and the moving and lifting times of shuttles reported in Table 5 are halved. To explore the impact of a specific parameter in

**Fig. 11.** Comparison of shuttle fleets.

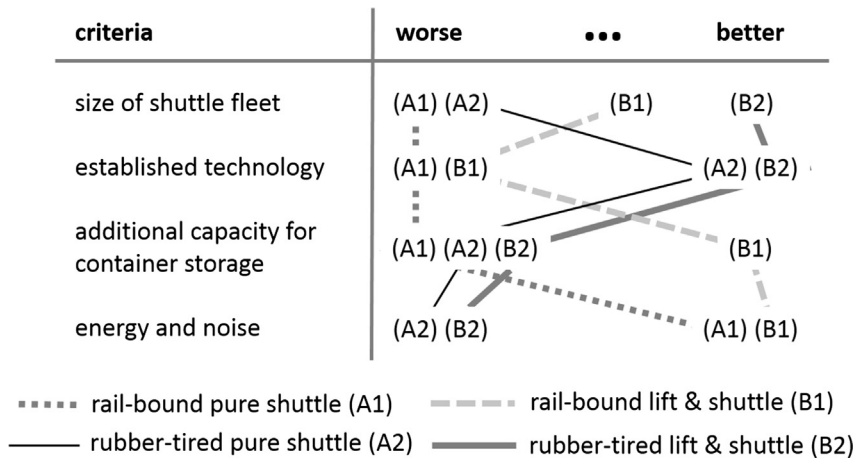


Fig. 12. Summary of comparison.

Table 9, we alter its value as stated in the third column and fix all other parameters to their default value (fourth column). For each scenario, we repeat instance generation 10 times, yielding to a total of 340 instances.

The results are summarized in Fig. 11 where we relate the required shuttle fleets of all four sorting systems to the varying parameter values applied during instance generation. The following conclusions can be drawn from these results:

- First of all, it can be concluded that the ranking of sorting systems with regard to the required fleet size remains identical within all four graphs: The rubber-tired lift & shuttle system requires the smallest shuttle fleet to timely supply containers. On average over all our instances this system comes by with just 2.02 shuttles. On the other end of the spectrum, we have the pure shuttle systems in their rubber-tired and rail-mounted version. Both require a considerably larger shuttle fleet, i.e., 13.55 and 13.99 vehicles on average, respectively. In between ranges the rail-mounted lift & shuttle system, which requires 4.82 vehicles on average. Obviously, the handover of boxes at the area borders slows down the sortation process, so that the shuttle fleet is larger than that of its rubber-tired counterpart. However, the autonomous pick-up and set-down of boxes considerably reduces the shuttle fleet compared to the pure shuttle systems.
- Note that in some cases the results of the rail-mounted lift & shuttle system are missing within Fig. 11. This is the case whenever the system is not able to timely deliver at least 90% of all boxes. Recall that adding more shuttles to the rail-mounted lift & shuttle system only increases system performance only up to a certain point. If there are too many vehicles per sorter lane the areas become too small and too many handovers impede a timely delivery. Thus, if there are (i) too much container moves to be handled within the 20 min time window, (ii) the moves are very long and often lead along the complete spread of the yard, or (iii) time windows are very tight, the rail-mounted lift & shuttle system is not able to reach the given level timely deliveries.
- The shuttle fleet required by the rubber-tired lift & shuttle system remains remarkably small (and stable) if the parameter values applied for instance generation increase. On the other hand, the number of shuttles required by the pure shuttle systems considerably increases if (i) the number of jobs $|J|$ to be handled, (ii) the length of moves L , or (iii) the length of time windows β increases. On the other hand, both systems barely profit from much faster shuttles (iv). Obviously, the long waiting times for the loading and unloading by cranes is the predominant factor that keeps the required shuttle fleets large. The lift & shuttle systems profit from faster shuttles with the rail-mounted version nearly reaching the performance its rubber tired counterpart if the shuttles are very fast and the handovers of boxes can quickly be executed.

We also altered the probability for container transfer α as stated in Table 9. This parameter is only of interest in the rail-mounted lift & shuttle system, as this is the only system requiring container transfers between the two rail tracks. However, the results did not show significant impact of α on the fleet size.

It can be concluded that two lift AGVs seem sufficient to handle representative workloads of rail-rail transshipment yards if a rubber-tired lift & shuttle system is selected. Due to the additional handling effort for the handover of containers the shuttle fleet in a rail-mounted lift & shuttle system is to be dimensioned about twice or three times as large. If there is a close synchronization between cranes and shuttles, i.e., the time windows β are very tight and unproductive waiting is not too long, three to four times more shuttles are required by the pure shuttle systems. On average over all our instances, however, their fleet sizes are to be dimensioned about seven times larger. These numbers give practitioners having to decide for a proper sorting system a general idea on the resulting investment costs. However, the size of the shuttle fleets are not the only relevant factor influencing the selection process. Fig. 12 summarizes the main selection criteria and the evaluation of the four sorting system.

AGVs and lift AGVs are well-established technologies applied in container ports since decades (Vis, 2006), whereas rail-mounted shuttles are new technological developments, which are rarely applied (Kreutzberger and Konings, 2016). An

exceptional feature of the rail-mounted lift & shuttle system is that the system can also be applied for long-term storage of containers. The container racks erected along the spread of the yard can not only be applied for short-term storage when supplying the cranes, but also storing boxes over a longer period of time. This way some hundred additional storage positions for containers are provided. Finally, rail-mounted vehicles reduce the rolling friction and, therefore, reduce energy consumption and noise emissions (for more details, e.g., see [Holmberg et al., 2012](#)). In light of these competing criteria, it can be concluded that selecting the right sorting system remains a delicate choice, which mainly influences the successful diffusion of hub-and-spoke networks in the railway system.

7. Conclusion

Modern rail-rail transshipment yards apply shuttle based sorting systems to relieve the gantry cranes from horizontal travel along the spread of the yard. The shuttles of the sorter transport containers along the yard between the inbound train's railcar a box arrives on and its dedicated outbound wagon. This way interference among cranes is avoided and they can concentrate on quickly transferring containers among trains. This paper compares four alternative sorting systems with regard to the size of the shuttle fleets required by each system to timely provide containers for the cranes. Specifically, we compare rubber-tired vehicles (i.e., AGVs and lift AGVs) with rail-mounted systems and differentiate whether pure shuttle systems are applied or if the shuttles are equipped with a lifting ability to autonomously pick up and set down containers. For each of the four systems suited scheduling procedures to determine the required fleet sizes for a given workload of container moves are developed. These procedures are applied in a comprehensive computational study. Our results reveal that the shuttle fleet required by the rubber-tired lift & shuttle system where lift AGVs are applied is the smallest. Considerably more shuttles are required by the pure shuttle systems, whereas the rail-mounted lift & shuttle system ranges somewhere in between.

Future research should reconsider our scheduling procedures for all four sorting systems. For both pure shuttle systems and the rubber-tired lift & shuttle system interference among shuttles should be integrated. This would allow a more precise quantification of the required shuttle fleets. For the rail-mounted lift & shuttle system an alternative shuttle control policy should be evaluated. In our system, shuttle interference is ruled out with certainty by introducing fixed areas each operated by a dedicated shuttle. The resulting handovers of containers at the area borders slow down the system. Without fixed areas an online control of shuttles avoiding collisions in realtime is required. This necessitates much more complex scheduling procedures steering the evasion moves of shuttles.

Appendix A. Solving rail-rubber-PSS by maximum matching in a bipartite graph

This appendix proves that rail-rubber-PSS, which determines the fleet size for the pure shuttle systems (see Section 2), can efficiently be solved to optimality by maximum matching in a bipartite graph. We reduce rail-rubber-PSS to this problem as follows:

Given an instance of rail-rubber-PSS, consider the bipartite graph $G = (V, E)$ with bipartition (P, Q) where nodes of both sets represent the job set J , so that $P = Q = J$. The set of edges connects a node $j \in P$ to a node $j' \in Q$, if and only if both nodes do not refer to the same job and job j' can be processed after j by the same shuttle, that is $a_j \geq d_j + r_{jj'}$. In the following we will show that a maximum matching M in graph G corresponds to a feasible solution to rail-rubber-PSS with minimal number of shuttles $Z(S)$.

We can think of a feasible solution to rail-rubber-PSS with $Z(S)$ shuttles as a set of $Z(S)$ chains of jobs, such that each chain represents the set of jobs successively processed by a specific shuttle. For each pair (j, j') of consecutive jobs in a chain choose $(j, j') \in E$ to be in M . Obviously, edges in M are non-adjacent since in the chains each job has at most one successor and at most one predecessor. For every shuttle $i = 1, \dots, Z(S)$ there will be one container without a predecessor, namely the first job the shuttle executes, and one job without a successor, namely the shuttle's last job. All other jobs performed by the shuttle (if any) must be consecutive and thus part of the matching. Therefore, the cardinality of the matching $|M|$ equals $n - Z(S)$, which is maximal if the number of shuttles $Z(S)$ is minimal.

Conversely, we can also construct a feasible solution to rail-rubber-PSS with a minimal shuttle fleet from a maximum matching M . Choose jobs corresponding to nodes in Q not covered by M as first jobs in the chains. Now attach job j' to the end of the chain having j as the currently last job if $(j, j') \in M$. Repeat this last step until all jobs are attached to a chain. The chains obtained constitute a feasible solution to the instance of rail-rubber-PSS. First, note that jobs are in the same chain only if they can be processed by the shuttle due to the construction of E . Second, each job is contained in exactly one chain due to the matching structure. Each job must be contained in at least one chain since each node $j \in Q$ either is not covered by M , which makes j the first container in a chain, or is covered by M which makes j the immediate successor of another job. Since no job can be its own successor, each container being the immediate successor of another job must be the (not necessarily immediate) successor of a job, which is first in a chain. Since every chain originates in a job $j \in Q$ not covered by M , the number of chains – and hence the number of shuttles $Z(S)$ – is minimal when as many jobs in Q are covered by M as possible, which is the case if the matching is maximum.

Table B.2

Notation for the rail-LSS-LP.

J	Set of container moves to be processed $J = \{1, 2, \dots, n\}$
R_j	Set of possible container routes of container j
S	Number of slots in the yard with $s = 1, \dots, S$
t	Index of sorter track with $t = 1, 2$
H	Set of handover slots with $H = \{h_1, h_2, \dots, h_m\} \subset \{1, \dots, S\}, h_1 < h_2 < \dots < h_m$ and $ H \geq \delta^t - 1 \forall t = 1, 2$
δ^t	Number of shuttles, with $i = 0, 1, \dots, \delta^t$, ordered according to their succession and assigned to sorter track t including dummy shuttle $i = 0$
$w_{t,h,h'}^r$	Workload (of loaded moves) caused by a container taking route r for a shuttle serving the yard area between two handover slots located in slots h and h' on track t
$x_{i,t,h}$	Binary variable: 1, if the handover slot between the areas of shuttles i and $i + 1$ on track t is in slot h , 0 otherwise
$y_{j,r}$	Binary variable: 1, if container j takes route r , 0 otherwise
$z_{i,t,h,h'}^{j,r}$	(Auxiliary) binary variable: $z_{i,t,h,h'}^{j,r} = x_{i-1,t,h} \cdot x_{i,t,h'} \cdot y_{j,r}$
c	(Auxiliary) continuous variable: maximum workload of all shuttles (makespan)

A maximum matching in a bipartite graph can be found in $\mathcal{O}(n^{2.38})$ (see Mucha and Sankowski, 2004). Thus, the runtime is polynomially bounded.

Appendix B. Mixed-integer program for holistic area sizing and container routing (rail-LSS)

This appendix provides a mixed-integer model that solves the holistic area sizing and container routing problem for our rail-mounted lift & shuttle system. It is applied to assess the performance of our decomposition approach solving both sub-problems in an iterative manner. Given all potential container routes and their resulting workloads per area, by applying the notation summarized in Table B.2 the holistic area sizing and container routing problem for the rail-mounted lift & shuttle system (rail-LSS-LP) consists of objective function (B.1) and constraints (B.2) and (B.12):

$$(\text{rail-LSS-LP}) \text{ Minimize } C(X, Y, Z, c) = c \quad (\text{B.1})$$

$$\text{subject to } c \geq \sum_{j \in J} \sum_{r \in R_j} \sum_{h \in H \cup \{1, S\}} \sum_{h' \in H \cup \{1, S\}} w_{t,h,h'}^r \cdot z_{i,t,h,h'}^{j,r} \quad \forall t = 1, 2, \quad i = 1, \dots, \delta^t \quad (\text{B.2})$$

$$\sum_{h \in H \cup \{1, S\}} x_{i,t,h} = 1 \quad \forall t = 1, 2, \quad i = 0, \dots, \delta^t \quad (\text{B.3})$$

$$\sum_{h \in H \cup \{1, S\}} h \cdot x_{i-1,t,h} < \sum_{h \in H \cup \{1, S\}} h \cdot x_{i,t,h} \quad \forall t = 1, 2, \quad i = 1, \dots, \delta^t \quad (\text{B.4})$$

$$x_{0,t,1} = 1 \quad \forall t = 1, 2 \quad (\text{B.5})$$

$$x_{\delta^t,t,S} = 1 \quad \forall t = 1, 2 \quad (\text{B.6})$$

$$\sum_{r \in R_j} y_{j,r} = 1 \quad \forall j \in J \quad (\text{B.7})$$

$$3 \cdot z_{i,t,h,h'}^{j,r} \leq x_{i-1,t,h} + x_{i,t,h'} + y_{j,r} \quad \forall t = 1, 2, \quad i = 1, \dots, \delta^t, \\ h, h' \in H \cup \{1, S\}, \quad j \in J, \quad r \in R_j \quad (\text{B.8})$$

$$z_{i,t,h,h'}^{j,r} \geq x_{i-1,t,h} + x_{i,t,h'} + y_{j,r} - 2 \quad \forall t = 1, 2, \quad i = 1, \dots, \delta^t, \\ h, h' \in H \cup \{1, S\}, \quad j \in J, \quad r \in R_j \quad (\text{B.9})$$

$$x_{i,t,h} \in \{0, 1\} \quad \forall t = 1, 2, \quad i = 0, \dots, \delta^t, \quad h \in H \cup \{1, S\} \quad (\text{B.10})$$

$$y_{j,r} \in \{0, 1\} \quad \forall j \in J, \quad r \in R_j \quad (\text{B.11})$$

$$z_{i,t,h,h'}^{j,r} \in \{0, 1\} \quad \forall t = 1, 2, \quad i = 1, \dots, \delta^t, \\ h, h' \in H \cup \{1, S\}, \quad j \in J, \quad r \in R_j \quad (\text{B.12})$$

Objective function (B.1) minimizes the maximum (loaded) workload of all shuttles, which is defined in (B.2). Constraints (B.3)–(B.6) deal with the area sizing problem and ensure that the assignment is well-defined (B.3), that shuttles are considered consecutively and each shuttle serves at least some part of the yard (B.4, the yard area of the first shuttle starts in the first slot (B.5), and the last shuttle's area ends in the last slot (B.6). (B.7) enforces that each container takes exactly one of its potential routes. In order to obtain a linear model, auxiliary variables $z_{i,t,h,h'}^{j,r}$ have to be introduced and defined. Their value is forced to $z_{i,t,h,h'}^{j,r} = x_{i-1,t,h} \cdot x_{i,t,h'} \cdot y_{j,r}$ by (B.8) and (B.9). Finally, (B.10)–(B.12) define the domains of the binary variables.

Appendix C. Determining potentially optimal container routes

Combining all possible changes towards the other sorter track via each handover slot leads to a huge number of possible routes each container may take. However, it can be shown that there always exists an optimal solution to the container routing

ing problem coming by without specific types of routes, so that the total number of routes can be reduced by the following two observations.

Depending on the relation of start and target slot each move has a general *move direction* either to the left or right. Note that a container having identical start and target slot cannot occur, since such a move would directly be executed by the gantry crane without accessing the sorter.

Theorem 3. *A container movement in opposite move direction can only relieve some shuttle if (i) it occurs either in the shuttle area neighboring the start slot and the start track or the area neighboring the target slot and the target track of a job. Furthermore, (ii) the distance between start (target) slot and the handover slot in opposite direction has to be shorter than that in move direction.*

Instead of a formal proof we explain the straightforward reasoning behind this theorem with the help of the examples depicted in Fig. C.1. Here, the movement opposite to the general move direction of partial container move (a) could potentially release shuttle $i = 1$ on track $t = 2$ (the bottom left) if this vehicle turns out as the bottleneck resource. Partial container move (b) fulfills condition (i) but not (ii), so that partial move (c) is always preferable for both affected shuttles (the right ones on top and bottom). Clearly, any other movement in opposite move direction inevitably requires the movement back and causes a loop (see move (d) of Fig. C.1), so that a release is not possible and it is always better to directly move on in move direction.

Furthermore, the following dominance rule can be applied to further reduce route sets R_j .

Dominance rule: A route r' is dominated by another route r and can, thus, be discarded from R_j , if they both refer to the same job and $w_{it}^r \leq w_{it}^{r'} \forall i = 1, \dots, \delta^t; t = 1, 2$, that is each shuttle has a lower or equal workload when executing r instead of r' .

Clearly, any optimal solution to rail-LSS-CR containing route r can simply swap r with r' and the workload of all shuttles cannot increase. Thus, there is always an optimal solution only consisting of non-dominated routes.

Combining both concepts, the number of routes may still become exponentially large, i.e., $|R_j|$ is in $\mathcal{O}(2^m)$. In the worst case, each job has to be moved between the leftmost and the rightmost shuttle area and the number of shuttles is large, so that in each handover slot successively visited a track change might be a suitable choice. However, in the average case this number should be much smaller, as is shown in the following example.

Example. Consider the example given in Fig. 8(a), where three containers have to be moved. All resulting potentially optimal (along with dominated) routes are depicted in Fig. C.2.

Appendix D. Computational complexity of the container routing problem

In this section, we show that our container routing problem minimizing the maximum workload for given shuttle areas (denoted as rail-LSS-CR) is NP-hard. We show this by a transformation from the PARTITION problem, which is well-known to be NP-complete (Garey and Johnson, 1979). The PARTITION problem is defined as follows: Given a set C of q positive integers c_j ($j = 1, \dots, q$) with $\sum_{j \in C} c_j = B$, does there exist a partition $C' \subseteq C$, such that $\sum_{j \in C'} c_j = B/2$?

The transformation to rail-LSS-CR is defined as follows. Consider a sorter having only a single shuttle per track $t = 1, 2$. The number of slots is $S = 2B + 1$ and we have $m = 2$ handover slots one at each end of the yard, so that $h_1 = 1$ and $h_2 = S$. We introduce $n = q$ container moves all originating at track $t = 1$ to be delivered to $t = 2$ with start and target slot $\alpha_j = B + c_j + 1$ and $\beta_j = B - c_j + 1$ for all $j \in J$. The shuttles move with one slot per time unit and take zero for the pick-up and set-down operations of containers. We will show that the answer to the instance of PARTITION is yes if and only if the minimum maximum workload for rail-LSS-CR is qB .

Let $J^1 \subseteq J$ be the set of containers handed over the left handover slot 1 and let $J^S = J \setminus J^1$, i.e., all other containers moved via the right handover slot. Workloads W_α and W_β of origin side shuttle and destination side shuttle, then, are:

$$W_\alpha = \sum_{j \in J^1} (B + c_j) + \sum_{j \in J^S} (B - c_j) = q \cdot B + \sum_{j \in J^1} c_j - \sum_{j \in J^S} c_j \quad (D.1)$$

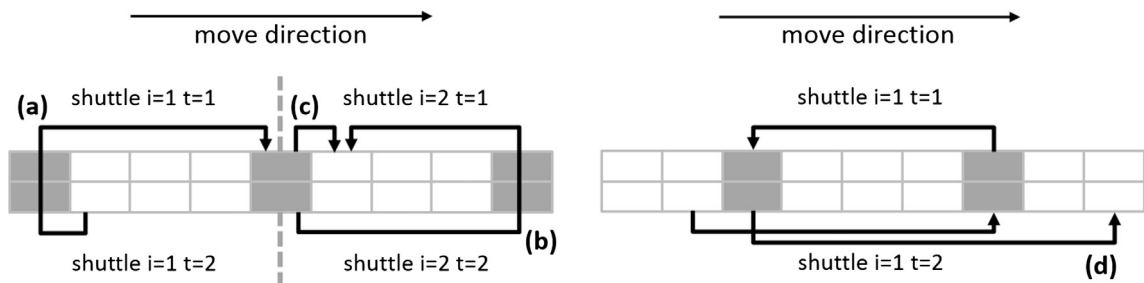


Fig. C.1. Container routes including a shuttle movement in opposite move direction.

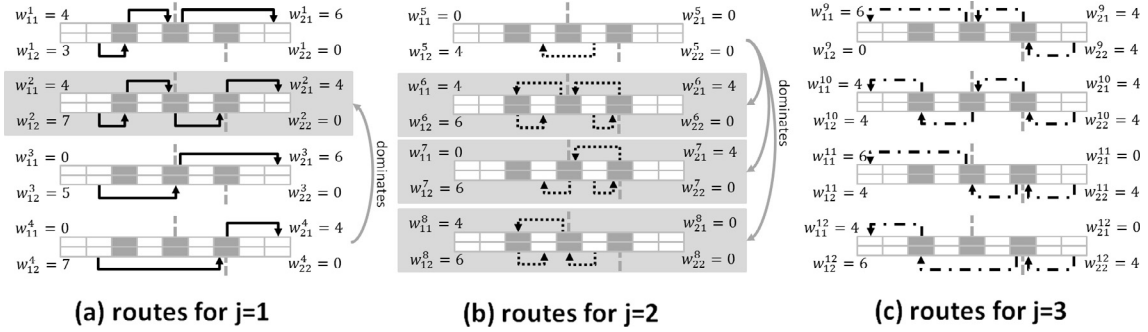


Fig. C.2. All potentially optimal container routes for the example of Fig. 8a.

$$W_\beta = \sum_{j \in J^1} (B - c_j) + \sum_{j \in J^2} (B + c_j) = q \cdot B - \sum_{j \in J^1} c_j + \sum_{j \in J^2} c_j \quad (D.2)$$

If the maximum workload is qB , then, $\sum_{j \in J^1} c_j - \sum_{j \in J^2} c_j \leq 0$ and $-\sum_{j \in J^1} c_j + \sum_{j \in J^2} c_j \leq 0$ must hold, which can only be fulfilled if $\sum_{j \in J^1} c_j = \sum_{j \in J^2} c_j$.

Appendix E. Lower bound for routing problem rail-LSS-CR

This appendix specifies a lower bound argument for rail-LSS-CR, i.e., the routing subproblem, which selects a route for each container through our rail-mounted lift & shuttle system. This lower bound is applied by our dynamic programming procedure and by beam search (see Section 4.3). Our simple lower bound argument resembles the famous parallel machine bound and simply divides the total workload by the number of shuttles. However, the workload of a container move is not fixed, but depends on the route each box takes. Therefore, we first have to determine the minimum total workload consisting of the times for the shuttles' travel plus all pick-up and set-down operations. Note that we target the total sum of workloads, but do not care of how this workload is distributed among the shuttles.

Algorithm 2. Determine the minimum workload for a container move

input: start and target slot and track of a container move j

```

1   $w_j^{\min} \leftarrow 0$ 
2  if start and target slot are within the same handover area then
3      if start and target slot are on the same track then
4          add time for a pick up and a set down operation to  $w_j^{\min}$ 
5          add the moving time from start to target slot to  $w_j^{\min}$ 
6      else if start and target slot are within the first handover area then
7          determine workload (un-/loading and moving) for paths using the right handover slot
8          add this workload to  $w_j^{\min}$ 
9      else if start and target slot are within the last handover area then
10         determine workload (un-/loading and moving) for paths using the left handover slot
11         add this workload to  $w_j^{\min}$ 
12      else
13         determine workload (un-/loading and moving) for paths using left and right handover slot
14         add the minimum to  $w_j^{\min}$ 
15      end
16  else
17      add time for a pick-up and a set-down operation to  $w_j^{\min}$ 
18      add the moving time from start to target slot to  $w_j^{\min}$ 
19      Determine the minimum number of additional transfers  $\lambda$  via Dijkstra
20      add  $\lambda \cdot (\text{time for a pick-up and a set-down operation})$  to  $w_j^{\min}$ 
21  end

return: minimum workload  $w_j^{\min}$ 

```

The minimum workload $w_j^{\min}$ per job j can be determined in $\mathcal{O}(m)$ by Algorithm 2. The basic idea behind this algorithm is to differentiate two basic cases:

- **Start and target slot of j are within the same shuttle area (area between two handover slots):** The workload can directly be calculated by adding the required (loaded) moving time and the needed times for pick-up and set-down operations. If the start track differs from the target track, the paths via the left and right handover slots have to be compared and the minimum defines the smallest workload.
- **Start and target slot of j are within different shuttle areas:** The shortest path includes the direct moving time from start to target slot, at least one pick-up and set-down operation each, and potentially additional transfers between vehicles. The number of these additional transfers can be determined by a special implementation of the Dijkstra-Algorithm for shortest paths in graphs (Dijkstra, 1959). Hereby, we construct a graph $G = (V, E, \omega)$, including a set of vertices V , a set of edges E and weights $\omega(e)$ for each $e \in E$. V contains a node for start and target slot each as well as two nodes for every handover slot (one for each track). E contains edge $e = (v, w)$ if the respective slots of v and w belong to the same handover area and v is located before w in target direction. The weight of an edge $e = (v, w)$ has the value $\omega(e) = 1$, if leaving slot v in direction of slot w requires an additional transfer because another shuttle has to be used and value $\omega(e) = 0$ otherwise. The sum of weights for edges on the shortest path from start to target slot equals the number of additional transfers.

As each node has a maximum of two predecessors (and successors), we can check the shortest path in a stagewise (i.e., stages equal handover slots) manner, by comparing only two values for upper and lower track each. Therefore, the Algorithm runs in $\mathcal{O}(m)$. Recall that m denotes the number of handover slots. Given these minimum workloads $w_j^{\min}$ for all jobs $j \in J$, which are without loss of generality assumed to be integer values, our lower bound LB amounts to:

$$LB = \left\lceil \frac{\sum_{j \in J} w_j^{\min}}{\delta^1 + \delta^2} \right\rceil. \quad (E.1)$$

Example. For the rail-LSS-CR instance given in Fig. 8(a) the lower bound amounts to $LB = \left\lceil \frac{11+4+14}{2+2} \right\rceil = 8$.

Since each job's minimum workload is to be determined computing LB is in $\mathcal{O}(nm)$.

When computing this bound in any other state $(W_{11}, \dots, W_{\delta^1 1}; W_{21}, \dots, W_{\delta^2 2})$ of DP but the initial one, some routes have already been fixed. In these cases, each shuttle's workload has first to be filled up to the state's maximum workload $Z(W_{11}, \dots, W_{\delta^1 1}; W_{21}, \dots, W_{\delta^2 2})$, before the remaining workload can be equally distributed among all shuttles:

$$LB(\dots) = Z(\dots) + \left\lceil \frac{\max \left\{ \sum_{j \in J, j' < j} w_j^{\min} - \sum_{i=1}^{\delta^1} \sum_{t=1}^2 (Z(\dots) - W_{it}); 0 \right\}}{\delta^1 + \delta^2} \right\rceil, \quad (E.2)$$

where $\dots$ and j' denote a shortcut for state $(W_{11}, \dots, W_{\delta^1 1}; W_{21}, \dots, W_{\delta^2 2})$ and the current stage the state belongs to, respectively. The minimum workloads per job have to be calculated only once before starting DP and also the aggregated workloads can be precalculated, so that calculating LB for each state requires merely an iteration through all shuttles and is, thus, in $\mathcal{O}(\delta)$.

Example. Consider state $(4, 6; 7, 0)$ of stage $j' = 2$ in the DP graph depicted in Fig. 9. The minimum workload of yet unscheduled job $j = 3$ amounts to $w_j^{\min} = 14$, so that after filling each shuttles workload up to the maximum workload there remain three workload units and $LB(4, 6; 7, 0) = 7 + \left\lceil \frac{3}{4} \right\rceil = 8$.

References

- Alicke, K., Arnold, D., 1998. Optimierung von mehrstufigen Umschlagssystemen. *Fördern und Heben* 8, 769–772.
- Alicke, K., 2002. Modeling and optimization of the intermodal terminal mega hub. *OR Spectrum* 24, 1–17.
- Ambrosino, D., Siri, S., 2015. Comparison of solution approaches for the train load planning problem in seaport terminals. *Transp. Res. Part E* 79, 65–82.
- Bae, H.Y., Choe, R., Park, T., Ryu, K.R., 2011. Comparison of operations of AGVs and ALVs in an automated container terminal. *J. Intell. Manuf.* 22, 413–426.
- Bertossi, A.A., Carraraesi, P., Gallo, G., 1987. On some matching problems arising in vehicle scheduling models. *Networks* 17, 271–281.
- Bontekoning, Y., Priemus, H., 2004. Breakthrough innovations in intermodal freight transport. *Transp. Plan. Technol.* 27, 335–345.
- Bostel, N., Dejax, P., 1998. Models and algorithms for container allocation problems on trains in a rapid transshipment shunting yard. *Transp. Sci.* 32, 370–379.
- Boysen, N., Fliedner, M., 2010. Cross dock scheduling: classification, literature review and research agenda. *Omega* 38, 413–422.
- Boysen, N., Fliedner, M., Kellner, M., 2010. Determining fixed crane areas in rail-rail transshipment yards. *Transp. Res. Part E* 46, 1005–1016.
- Boysen, N., Jaehn, F., Pesch, E., 2011. Scheduling freight trains in rail-rail transshipment yards. *Transp. Sci.* 45, 199–211.
- Boysen, N., Fliedner, M., Jaehn, F., Pesch, E., 2012. Shunting yard operations: theoretical aspects and applications. *Eur. J. Oper. Res.* 220, 1–14.
- Boysen, N., Jaehn, F., Pesch, E., 2012. New bounds and algorithms for the transshipment yard scheduling problem. *J. Sched.* 15, 499–511.

- Boysen, N., Flidner, M., Jaehn, F., Pesch, E., 2013. A survey on container processing in railway yards. *Transp. Sci.* 47, 312–329.
- Boysen, N., Emde, S., 2014. Scheduling the part supply of mixed-model assembly lines in line-integrated supermarkets. *Eur. J. Oper. Res.* 239, 820–829.
- Carlo, H.J., Vis, I.F., Roodbergen, K.J., 2014. Transport operations in container terminals: literature overview, trends, research directions and classification scheme. *Eur. J. Oper. Res.* 236, 1–13.
- Delta Air Lines, 2013. <<http://news.delta.com/index.php?s=18&cat=47>> (last access: May 2015).
- Deutsche Bahn, 2016. <<https://bauprojekte.deutschebahn.com/media/projects/3104/docs/BR-Lehrte-MegaHub-Infrastrukturprojekte-201612.pdf>> (last access: January 2017).
- Dijkstra, E.W., 1959. A note on two problems in connexion with graphs. *Numer. Math.* 1, 269–271.
- Garey, M.R., Johnson, D.S., 1979. *Computers and Intractability*. New York.
- Gottwald, 2016. <http://www.demagcranes-ag.de/files/content/sites/deutschland/files/Products/Hafentechnologie/Automated_Guided_Vehicle/AGV_Brochure_German%5B1%5D.pdf> (last access: January 2017).
- Guo, P., Cheng, W., Zhang, Z., Zhang, M., Liang, J., 2013. Gantry crane scheduling with interference constraints in railway container terminals. *Int. J. Comput. Intell. Syst.* 6, 244–260.
- Holmberg, K., Andersson, P., Erdemir, A., 2012. Global energy consumption due to friction in passenger cars. *Tribol. Int.* 47, 221–234.
- Kellner, M., Boysen, N., Flidner, M., 2012. How to park freight trains on rail-rail transshipment yards: the train location problem. *OR Spectrum* 34, 535–561.
- Kolen, A.W., Lenstra, J.K., Papadimitriou, C.H., Spiessma, F.C., 2007. Interval scheduling: a survey. *Nav. Res. Logist.* 54, 530–543.
- Kreutzberger, E., Konings, R., 2016. The challenge of appropriate hub terminal and hub-and-spoke network development for seaports and intermodal rail transport in Europe. *Res. Transp. Bus. Manage.* 19, 83–96.
- Lowerre, B., 1976. *The Harpy Speech Recognition System* PhD thesis. Carnegie Mellon University.
- Martinez, F.M., Gutierrez, I.G., Oliveira, A.O., Bedia, L.M.A., 2004. Gantry crane operations to transfer containers between trains: a simulation study of a Spanish terminal. *Transp. Plan. Technol.* 27, 261–284.
- Meyer, P., 1998. *Entwicklung eines Simulationsprogramms für Umschlagterminals des Kombinierten Verkehrs* PhD thesis. Universität Hannover.
- Mucha, M., Sankowski, P., 2004. Maximum matchings via Gaussian elimination. In: *Proc. 45th IEEE Symp. Foundations of Computer Science*, pp. 248–255.
- Olariu, S., 1991. An optimal greedy heuristic to color interval graphs. *Inform. Process. Lett.* 37, 21–25.
- Rotter, H., 2004. New operating concepts for intermodal transport: the mega hub in Hanover/Lehrte in Germany. *Transp. Plan. Technol.* 27, 347–365.
- Steenken, D., Voß, S., Stahlbock, R., 2004. Container terminal operation and operations research – a classification and literature review. *OR Spectrum* 26, 3–49.
- Trip, J.J., Bontekoning, Y.M., 2002. Integration of small freight flows in the intermodal transport system. *J. Transp. Geogr.* 10, 221–229.
- Unlu, Y., Mason, S.J., 2010. Evaluation of mixed integer programming formulations for non-preemptive parallel machine scheduling problems. *Comput. Ind. Eng.* 58, 785–800.
- Vis, I.F., 2006. Survey of research in the design and control of automated guided vehicle systems. *Eur. J. Oper. Res.* 170, 677–709.
- Vis, I.F., Harika, I., 2004. Comparison of vehicle types at an automated container terminal. *OR Spectrum* 26, 117–143.
- Yang, C.H., Choi, Y.S., Ha, T.Y., 2004. Simulation-based performance evaluation of transport vehicles at automated container terminals. *OR Spectrum* 26, 149–170.
- Zhen, L., Chew, E.P., Lee, L.H., 2011. An integrated model for berth template and yard template planning in transshipment hubs. *Transp. Sci.* 45, 483–504.